



第九章 串行总线接口技术

刘剑锋

ljf@csu.edu.cn





本章主要内容

9.1 SPI串行总线接口技术

9.2 I²C总线接口技术

9.3 单总线(1-wire)接口

第9章 串行总线接口技术

由于数据的串行传输连线少，因而采用串行总线扩展技术可以使系统的硬件设计简化，系统的体积减小，同时，系统的更改和扩充更为容易。

目前，单片机应用系统中常用的串行扩展总线有：**I2C(Inter IC BUS)总线、SPI(Serial Peripheral Interface)总线、Microwire总线及单总线(1-Wire BUS)。**

串行扩展总线的应用是单片机目前发展的一种趋势。AT89系列单片机利用自身的通用并行线可以模拟多种串行总线时序信号，因此可以充分利用各种串行接口芯片资源。

本章主要介绍I2C总线、SPI总线及单总线(1-Wire BUS)的基本知识、常用的串行总线接口器件及和单片机的接口应用。

9.1 SPI串行总线接口技术

9.1.1 SPI串行总线简介

SPI接口的全称是“Serial Peripheral Interface”,意为串行外围接口,是Motorola首先在其MC68HCXX系列处理器上定义的。

SPI总线系统是一种同步串行外设接口,它可以使MCU与各种外围设备以串行方式进行通信以交换信息。SPI总线系统可直接与各个厂家生产的多种标准外围器件直接接口,该接口一般包括以下四种信号:

- ① MOSI - 主器件数据输出,从器件数据输入
- ② MISO - 主器件数据输入,从器件数据输出
- ③ SCLK - 时钟信号,由主器件产生
- ④ /ss- 从器件使能信号,由主器件控制



SPI接口是在CPU和外围低速器件之间进行同步串行数据传输, 在主器件的移位脉冲下, 数据按位传输, 高位在前, 低位在后, 为全双工通信, 数据传输速度总体来说比I2C总线要快, 速度可达到几Mbps。

对于大多数不带SPI串行总线接口的AT89系列单片机来说, 可以使用软件来模拟SPI的操作, 包括串行时钟、数据输入和数据输出。

9.1.2 SPI串行接口A/D转换器TLC549及其软硬件设计

TLC549是美国德州仪器公司生产的8位串行A/D转换器芯片，通过SPI接口与单片机连接，从CLK输入的频率最高可达1.1MHz。

TLC549具有4MHz的片内系统时钟，片内具有采样保持电路，A/D转换时间最长 $17\mu\text{s}$ ，最高转换速率为40 000次/s。

TLC549的电源范围为+3V- +6V，功耗小15Mw，总失调误差最大为 $\pm 0.5\text{LSB}$ ，适用于电池供电的便携式仪表及低成本高性能的系统中。

1. 引脚功能

TLC549有8个引脚，如图9-1所示。各引脚功能说明如下：

REF+： 正基准电压输入端， $2.5V \leq \text{REF+} \leq V_{CC} + 0.1V$ 。

REF-： 负基准电压输入端， $-0.1V \leq \text{REF-} \leq 2.5V$ ，且要求 $\text{REF+} - \text{REF-} \geq 1V$ 。在要求不高时，也可将REF-接地，REF+接VCC。

AIN： 模拟信号输入端， $0 \leq \text{AIN} \leq V_{CC}$ ，当 $\text{AIN} \geq \text{REF+}$ 时，转换结果为全“1”(FFH)， $\text{AIN} \leq \text{REF-}$ 时，转换结果为全“0”(00H)。

/CS： 芯片选择输入端，低电平有效。

DO： 数据串行输出端，输出时高位在前，低位在后。

CLK： 外部时钟输入端，最高频率可达1.1MHz。

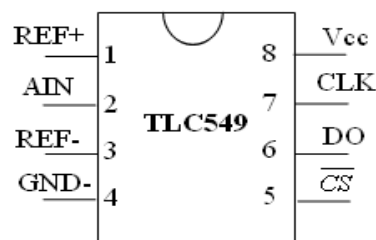


图9-1 TLC549引脚图

2. TLC549的时序

TLC549的时序如图9-2所示。 $\overline{\text{CS}}$ 变为低电平时，TLC549芯片被选中，同时从D0端输出前次转换结果的最高有效位A7；

接着自CLK端输入 8 个外部时钟信号，前 7 个CLK信号输出上次转换结果的A6-A7位。

在第 4 个CLK信号由高至低的跳变之后，片内采样/保持电路对输入模拟量采样开始，第 8 个CLK信号的下降沿使片内采样/保持电路进入保持状态并启动本次A/D开始转换。

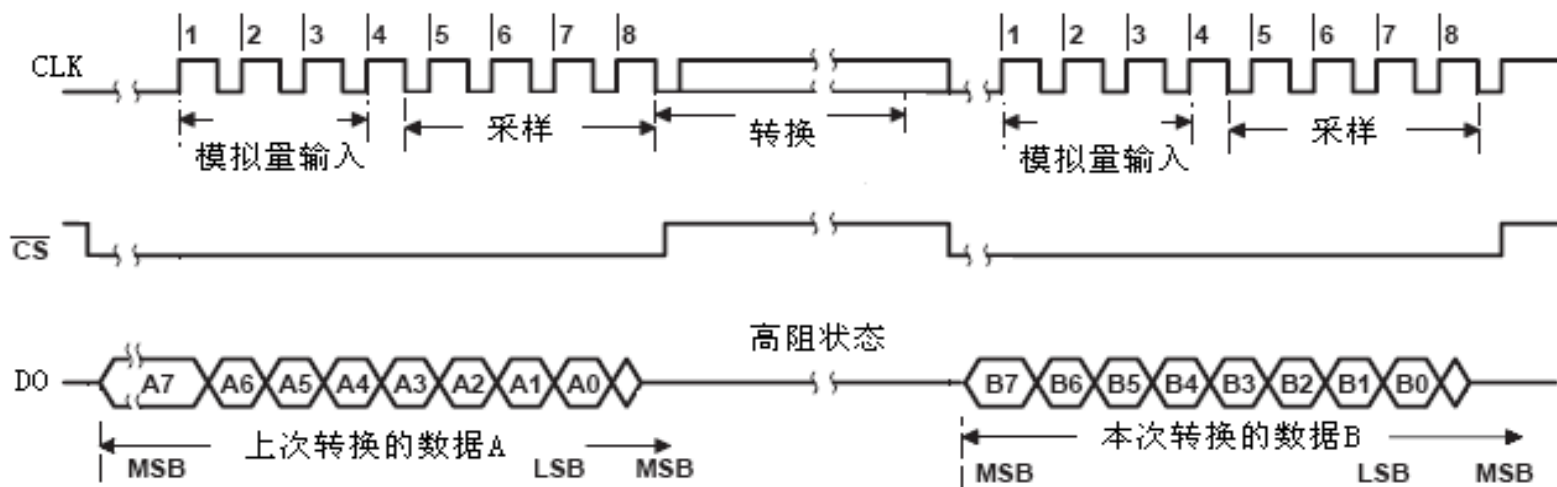


图9-2 TLC549的时序

TLC549没有启动控制端，只要读走前一次数据后马上就进行新的转换，转换完成后就进入保持状态，转换时间为36个系统时钟周期，最大为17 μ S。没有转换完成标志信号，只要采用延时操作即可控制每次读取数据的操作。

3. TLC549与单片机的接口

TLC549与单片机的连接如图9-3所示。采用P1.0~P1.2连接TLC549的串行接口。

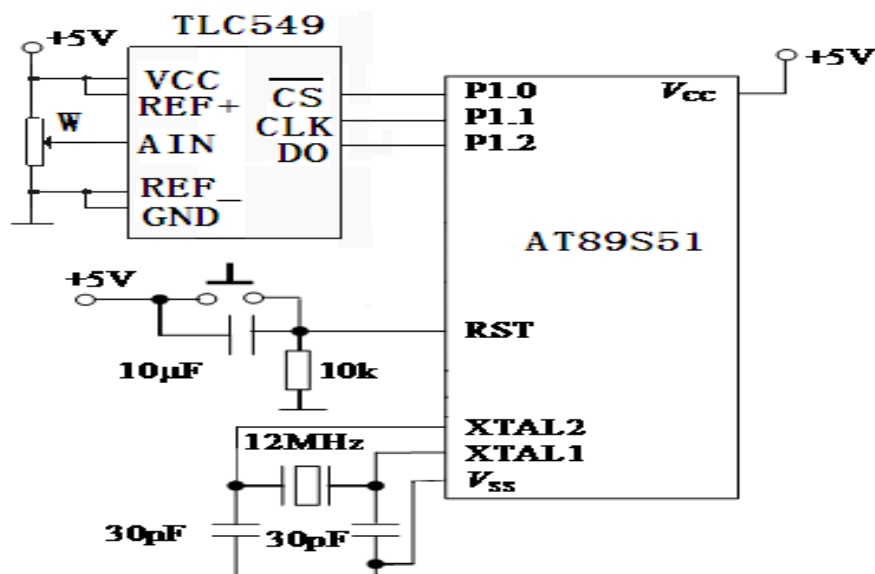


图9-3 TLC549与单片机的硬件连接

A/D转换的汇编语言程序:

DO BIT P1.2

CLK BIT P1.1

CS BIT P1.0

...

TLC549_AD: CLR A ; **TLC549 A/D转换子程序, 转换结果在A中**

CLR CLK

MOV R5,#08H

CLR CS ; **选中TLC549**

LOOP: SETB CLK ; **产生时钟**

NOP

NOP

NOP

NOP

MOV C,DO ; **读取A/D转换的一位数据**

RLC A ; **左移进入A**

CLR CLK



NOP

NOP

DJNZ R5,LOOP ; 判8次数据是否读完

SETB CS

SETB CLK

RET

A/D转换的C语言程序:

```
sbit DO = P1^2
sbit CLK = P1^1
sbit CS = P1^0
bdata Unsigned char addata;
sbit adin0 = addata ^0;
...
unsigned char TLC549 _ad(void)  /*A/D转换程序*/
{
    unsigned char i;
    Clk=0;
    CS=0;                      /*令CS为低选中TLC549*/
    _nop_();
```



```
for(i=0;i<8;i++)          /*循环读取8位A/D转换结果*/
{ CLK =1;                 /*令CLK引脚为高，产生时钟*/
  delay( );               /*延时*/
  adin0 =DO;              /*读取A/D转换后数据线的一位数据*/
  addata = addata <<1;    /*左移一位，先读取为高位，后读
为低位*/
  CLK=0;                  /*令CLK恢复为0*/
  _nop_( );
  _nop_( );
}
return addata;            /*返回A/D转换值*/
}
```

```
void delay ( )
{ unsigned char i;
  for(i=0;i<20;i++)
  }
```

4. 简易数字电压表的设计举例

利用TLC549 A/D转换器设计一个简易数字电压表，用4位LED显示器将被测电压显示出来，测量范围为0V-5.000V。将TLC549的、CLK、DO接到单片机的三条I/O口线，REF+、REF-直接接到V_{cc}、GND，模拟输入AIN接电位器的中心抽头，调节电位器即可改变被测输入电压值，硬件连接如图9-4所示。

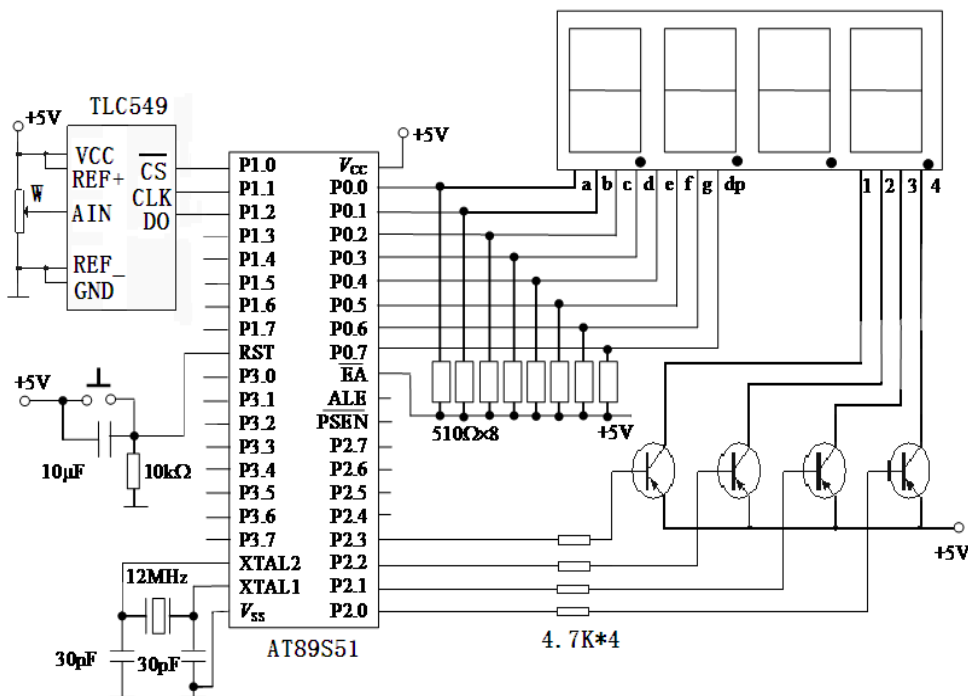


图9-4 简易数字电压表硬件连接图

软件设计的基本思路:

程序首先通过调用TLC549_ad(), 读取A/D转换结果存入addata, 然后按公式 $u = \text{addata} / 255 \times 5000$ (mV) 计算电压值, 再将u转换为4位BCD码送显示缓冲区, 并调用显示程序disp_ad()将其转换为字型码显示出来, 显示格式为x. xxxx, 单位为v。

程序请参照课本.

9.1.3 SPI串行接口D/A转换器TLC5615软硬件设计

TLC5615 是SPI接口的10位电压输出的D/A转换器，通过3根串行总线就可以完成10 位数据的串行输入，易于和工业标准的微处理器或单片机接口，适用于电池供电的测量仪表、移动电话以及工业控制场合。

其主要特点如下：

- 5V单电源工作；
- 3 线串行接口；
- DAC 输出的最大电压为2 倍基准输入电压；
- 上电时内部自动复位，确保可以重复启动；
- 功耗低，最大功耗为1.75mW。

1. TLC5615的内部结构和引脚功能

TLC5615的内部结构如图9-5所示，内部包含：

- 一个电压跟随器为参考电压端REFIN提供高输入阻抗；
- 10位DAC×2 电路提供最大值为2倍于REFIN 的输出；
- 一个16 位移位寄存器，接受串行移入的二进制数，并且有一个级联的数据输出端DOUT；
- 并行输入输出的10位DAC寄存器，为10位DAC电路提供待转换的二进制数据。

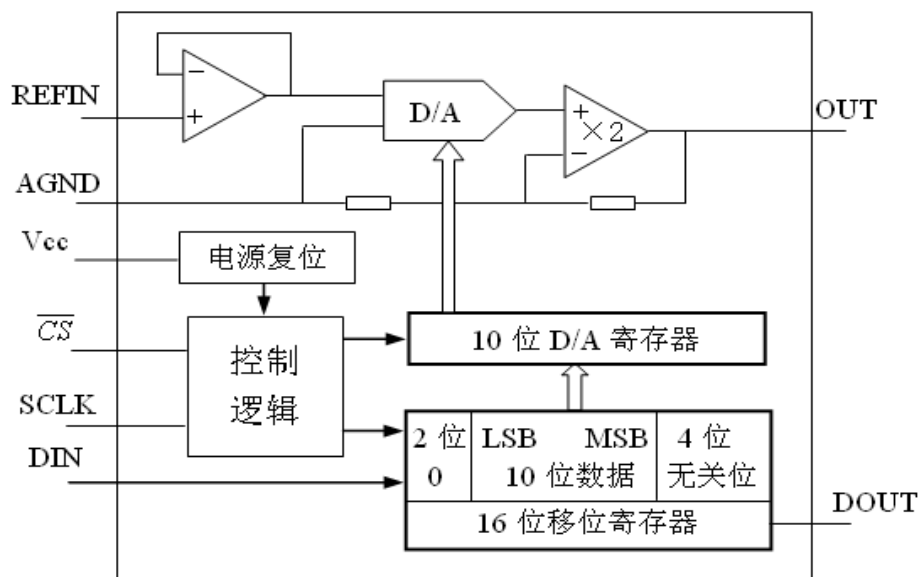


图9-5 TLC5615的内部结构

TLC5615的引脚如图9-6所示，各引脚功能如下：

DIN： 串行二进制数输入端

SCLK： 串行时钟输入端

/CS： 芯片选择，低有效

DOUT： 菊花链的串行数据输出端（用于多芯片的级联）

REFIN： 基准电压输入端

OUT： DAC 模拟电压输出端

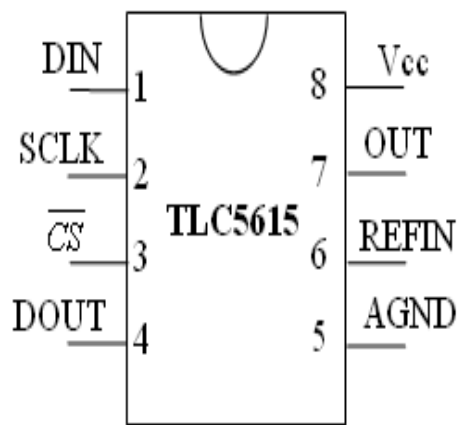


图9-6 TLC5615的引脚

2、TLC5615的时序

TLC5615的时序如图9-7所示。/CS当为低电平时，在每一个SCLK时钟的上升沿从DIN 引脚移入一位数据，高位在前，低位在后。经16个时钟后，/CS的上升沿将16 位移位寄存器的10 位有效数据锁存到10位DAC寄存器，供DAC电路进行转换。

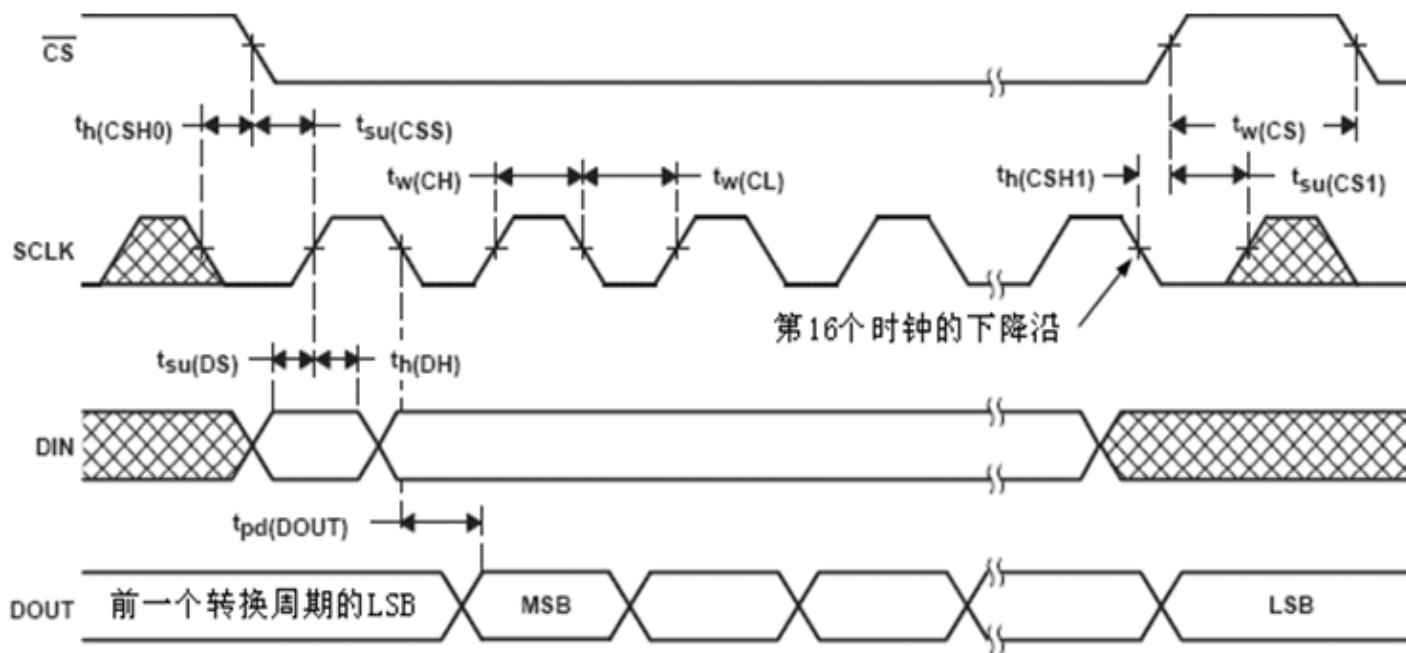


图9-7 TLC5615的时序

16位数据的高4位和低2位不会被转换，待转换数据输入的格式见表9-1：

表9-1 D/A转换数据输入格式

输入序号	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
输入数据	×	×	×	×	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0	0	0

设n为待转换的数字量，VREFIN为基准输入电压，则转换后的输出电压：

$$V_{OUT}=2 \times V_{REFIN} \times n/1024$$

3. TLC5615与单片机的接口

TLC5615与单片机的硬件如图9-7所示，将TLC5615的SCLK、CS、DIN分别与单片机的P1.0、P1.1、P1.2相连，基准电压接+5V。

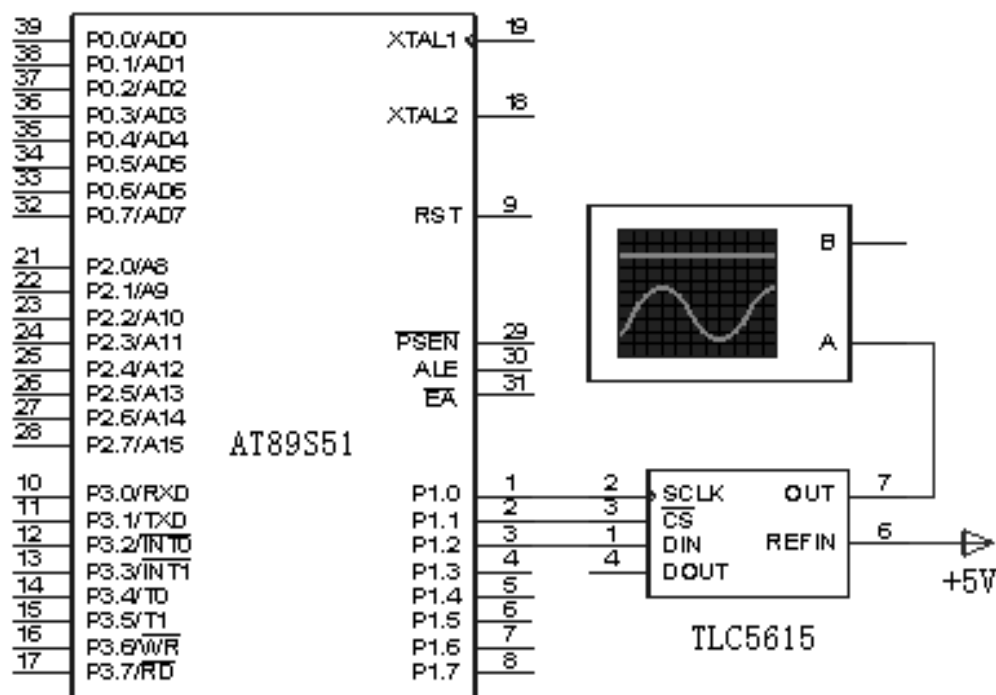


图9-8 TLC5615与单片机的硬件连接

D/A转换汇编语言程序:

设要转换的数据放在R7R6中，R7为高8位，R6低8位

DIN BIT P1.2 ;引脚定义

CS BIT P1.1

SCLK BIT P1.0

...

TLC5615_DA:CLR C ;将R7R6中数据左移2位(16位数据的最低2位添00)

RLC R6

RLC R7

CLR C

RLC R6

RLC R7

SETB CS ;初始化片选信号为高

CLR SCLK ;初始化时钟为低

CLR DIN ;D/A数据线置低

CLR CS ;选中TLC5615，开始启动D/A

MOV R5, #16 ;将16位数据从DIN端移进内部的16位移位寄

存器



LOOP: RLC R6 ;R7R6中数据左移一位，最高位进入CY

RLC R7

MOV DIN, C ;将数据送到DIN引脚

SETB SCLK ; 送时钟

NOP

NOP

NOP

CLR SCLK;

NOP

NOP

NOP

DJNZ R5, LOOP

**SETB CS ;D/A片选拉高，10 位有效数据锁存到
DAC寄存器,开始转换**

RET

D/A转换C语言程序:

sbit DA_clk=P1^0; /*引脚定义*/

sbit DA_cs=P1^1;

sbit DA_in=P1^2;

...

void delay_s(unsigned char n) /*延时*/

{ unsigned char i;

for(i=0; i<n; i++);

}

void TLC5615_DA_conver(unsigned int DA_data) /* D/A转换程序*/

{

unsigned char i;

DA_data = DA_data <<2; /*将数据左移2位(最低2位添00) */

DA_cs=1; /*初始化片选信号为高*/

DA_clk=0; /*初始化时钟为低*/

DA_in=0; /*D/A数据线置低*/

DA_cs=0; /*选中TLC5615, 开始启动D/A*/

for(i=0; i<16; i++) /*将16位数据从DIN端移进内部的16位移位寄存器*/



```
{
    DA_data = DA_data <<1;    /*左移一位，最高位进入CY*/
    DA_in=CY; /*将数据送到DIN引脚*/
    DA_clk=1; delay_s(0x02); /*送时钟*/
    DA_clk=0; delay_s(0x02);
}
DA_cs=1;                /*片选拉高，10 位数据锁存到DAC寄存器,开始转
换*/
delay_s(0x20);
}
```

4. 用TLC5615设计简易信号发生器

图9-9是用TLC5615设计简易信号发生器的硬件接口电路。系统实现上电后产生矩形波，当S1按下时产生正弦波，当S2按下时产生三角波。

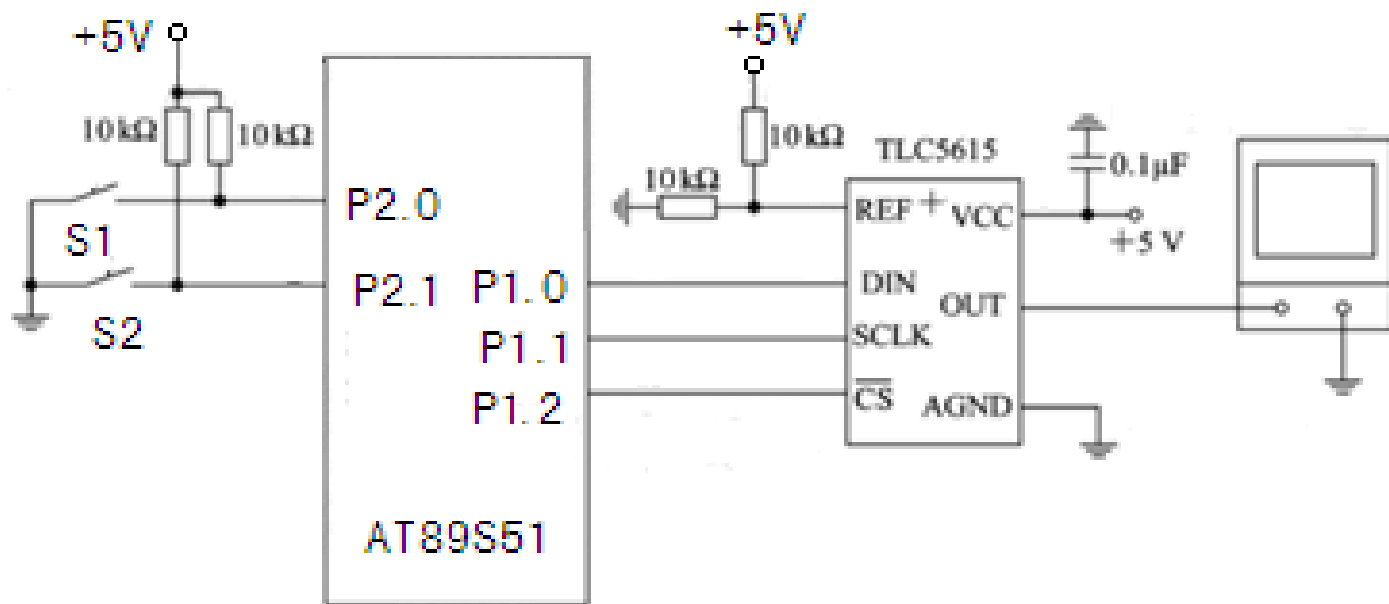


图9-9简易信号发生器的硬件接口电路

9.2 I²C总线接口技术

I2C总线是PHILIPS公司推出的一种高性能芯片间简单、双向二线制同步串行总线，数据传输时只需两根信号线，一根是双向数据线SDA，另一根是时钟线SCL，所有连接到I2C总线上的串行器件，其数据线都连接到总线的SDA上，时钟线则连接到总线的SCL上。

9.2.1 I2C总线简介

1. I2C总线的主要特点有：

总线只有两根线，即串行时钟线（SCL）和串行数据线（SDA），这在设计中大大减少了硬件接口。



每个连接到总线上的器件都有一个用于识别的器件地址，器件地址由芯片内部硬件电路和外部地址引脚同时决定，避免了片选线的连接方法，并建立简单的主从关系，每个器件既可以作为发送器，又可以作为接收器。

同步时钟允许器件以不同的波特率进行通信。

同步时钟可以作为停止或重新启动串行口发送的握手信号。

串行的数据传输位速率在标准模式下可达100kbit/s，快速模式下可达400kbit/s，高速模式下可达3.4Mbit/s。

连接到同一总线的集成电路数只受400pF的最大总线电容的限制。

2. I2C总线系统结构

I2C总线的系统结构如图9-10所示。采用I2C总线标准的器件均并联在总线上，内部都有I2C接口电路，用于实现和I2C总线的连接。I2C总线上的每一个从器件均有一个唯一的地址，用于识别不同的器件。

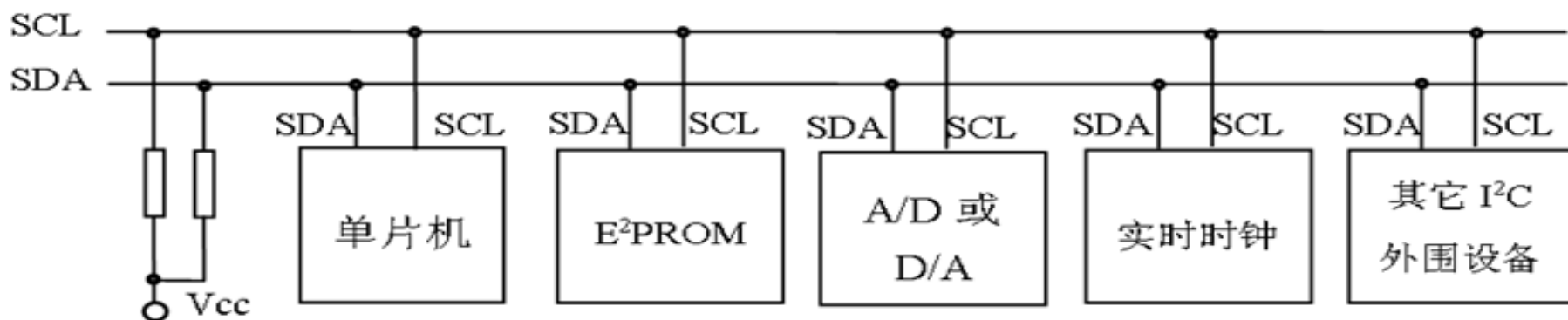


图9-10 I2C总线的系统结构

3 . I2C总线的工作时序

当I2C总线没有进行信息传送时，数据线（SDA）和时钟线（SCL）都为高电平时。当主控制器向某个器件传送信息时，首先应向总线送开始信号，然后才能传送信息，当信息传送结束时应送结束信号，如图9-11所示。

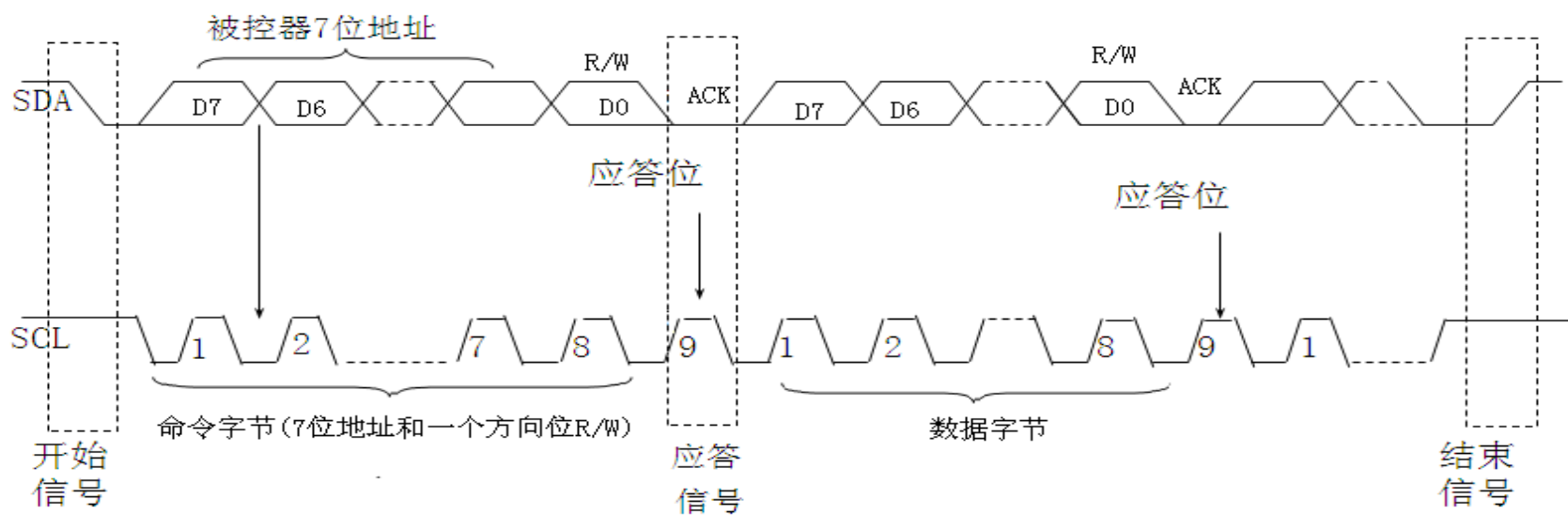


图9-11 I2C总线的工作时序

开始信号和结束信号规定如下：

开始信号：SCL为高电平时，SDA由高电平向低电平跳变，开始传送数据。

结束信号：SCL为高电平时，SDA由低电平向高电平跳变，结束传送数据。

开始信号和结束信号之间传送的是信息，信息的字节数没有限制，但每个字节必须为8位，高位在前，低位在后。

数据线SDA上每一位信息状态的改变只能发生在时钟线SCL为低电平的期间，因为SCL高电平期间SDA状态的改变已经被用来表示开始信号和结束信号。

每个字节后面必须接收一个应答信号（ACK），ACK是从控制器在接收到8位数据后向主控制器发出的特定的低电平脉冲，用以表示已收到数据。

主控制器接收到应答信号（ACK）后，可根据实际情况作出是否继续传递信号的判断。若未收到ACK，则判断为从控制器出现故障。

主控制器每次传送的信息的第一个字节必须是器件地址码，第二个字节为器件单元地址，用于实现选择所操作的器件的内部单元，从第三个字节开始为传送的数据。其中器件地址码格式如表9-2所示。

表9-2 I2C总线上器件地址码格式

D7	D6	D5	D4	D3	D2	D1	D0
器件类型码				片选			R/W



I2C总线上的每一个从器件均有一个唯一的地址，每次主器件发出起始信号后，必须接着发出一个字节的器件地址，以选取挂在总线上的某一从器件并控制总线的传输方向。

器件类型码：表示器件的类型，出厂时根据PHILIPS公司的I2C规程确定，比如对于24C02芯片来说，器件标识地址为1010。

片选地址：当总线上有片同类器件时，只有器件引脚A2~A0的电平与器件地址中A2~A0同相的器件才能被选中。R/W操作控制位，为1表示读操作，为0表示写操作。

说明：器件地址只表明选择挂在总线的哪一个器件及数据传送方向，而器件内部的地址是由编程者在传送第一个数据时指定的，即第一个数据为器件内的子地址。

4. I2C总线读、写操作

(1) 当前地址读

该操作将从所选器件当前地址执行读操作，读的字节数不指定，格式见表9-3所示

表9-3 当前地址读操作格式

S	控制码 (R/W=1)	A	数据1	A	数据2	A	P
---	-------------	---	-----	---	-----	---	---

(2) 指定单元读

该操作将从所选器件指定地址读，读的字节数不指定，格式见表9-4所示。

表9-4 指定地址读操作格式

S	控制码 (R/W=0)	A	器件单元地址	A	S	控制码 (R/W=1)	A	数据1	A	数据2	A	P
---	-------------	---	--------	---	---	-------------	---	-----	---	-----	---	---

(3) 指定单元写

该操作将从所选器件指定地址写，写的字节数不指定，格式见表9-5所示。

表9-5指定地址写操作格式

S	控制码 (R/W=0)	A	器件单元 地址	A	数据1	A	数据2	A	P
---	-------------	---	---------	---	-----	---	-----	---	---

其中：S表示开始信号，A表示应答信号，P表示结束信号。

9.2.2 用I/O口模拟I2C总线操作子程序

目前不少的单片机内部集成了I2C总线接口，如MCS-51系列的8XC550、8XC552、8XC571等，低价位的单片机内部虽然没有集成I2C总线接口，但可以通过软件实现I2C总线操作。

假设采用89S51单片机，晶振频率为12MHz，即机器周期为1 μ s，使用P1.0作为数据线SDA， P1.1作为时钟线SCL。

```
SCL BIT P1.1                ; 汇编语言定义端口
SDA BIT P1.0
sbit SDA=P1^0;              //C语言定义端口
sbit SCL=P1^1;
bit ack;                    //应答标志位，有应答为1，无应答为0
#define DELAY5US _nop_();_nop_();_nop_();_nop_();_nop_();
```

根据I2C总线数据传送的典型信号时序要求，可用单片机I/O口线产生起始信号、停止信号、应答信号、非应答信号等。汇编语言和C语言程序如下：

1.产生起始信号 S

汇编语言程序:

START:SETB SDA	;发送起始条件数据信号
SETB SCL	;发送起始条件的时钟信号
NOP	
NOP	
NOP	
NOP	
CLR SDA	;发送起始信号（SCL为高，SDA发生由高到低）
NOP	
NOP	
NOP	
NOP	
NOP	
CLR SCL	
RET	



C语言程序:

void start()

{

SDA=1; /*将SDA、SCL置为1 */

SCL=1;

DELAY5US; /*延时5us*/

SDA=0; /*SCL为高时，SDA由高变低*/

DELAY5US;

SCL=0; /*SCL变低，准备发送或接收数据 */

}

2.产生停止信号 S

汇编语言程序:

STOP: CLR SDA

SETB SCL

NOP

NOP

NOP

NOP

NOP

SETB SDA

SDA发生由低到高)

NOP

NOP

NOP

NOP

CLR SCL

CLR SDA

RET

;发送停止条件的数据信号

;发送停止条件的时钟信号

;发送I2C总线停止信号（SCL为高，



C语言程序：

void stop()

{ SDA=0; /*将SDA清0， SCL置1 */

SCL=1;

DELAY5US;

**SDA=1; /*当SCL为高电平时， SDA由低变
高 */**

DELAY5US;

SCL=0;

}

3.发送应答信号 ACK

汇编语言程序:

MACK: CLR SDA

SETB SCL

NOP

NOP

NOP

NOP

NOP

CLR SCL

SETB SDA

RET

;发送应答信号MACK



C语言程序:

```
void ack(void)      /*产生应答信号*/  
{ SDA=0;           /*SDA先清0，发应答信号 */  
  SCL=1;            /*SCL由低变高，产生一个时钟*/  
  DELAY5US;         /*延时5us */  
  SCL=0;            /*SCL变低，以便继续接收*/  
  SDA=1;  
}
```

4.发送非应答信号NACK

汇编语言程序：

NACK: SETB SDA ;发送应答信号NACK

SETB SCL

NOP

NOP

NOP

NOP

NOP

CLR SCL

CLR SDA

RET



C语言程序：

```
void nack(void)
```

```
{
```

```
    SDA=1;           /*DA先置1，发非应答信号 */
```

```
    SCL=1;           /*SCL由低变高，产生一个时钟*/
```

```
    DELAY5US;
```

```
    SCL=0;           /*时钟线SCL恢复到低电平*/
```

```
    SDA=0;
```

```
}
```


5. 应答检测子程序**CACK**

汇编语言程序(F0=1通信失败):

```
CACK: SETB SDA           ;发送应答信号CACK  
      SETB SCL  
      CLR F0  
      MOV C, SDA  
      JNC CEND  
      SETB F0  
CEND: CLR SCL  
      RET
```



C语言程序：

```
void cack(void)
```

```
{ SDA=1;          /*SDA先置1，发非应答信号 */  
  SCL=1;          /*SCL由低变高，产生一个时钟*  
  DELAY5US;  
  ack=0;  
  if(SDA=1)  
    ack=1;  
  SCL=0;          /*时钟线SCL恢复到低电平*/  
}
```

6.向I2C总线发送一个字节

汇编语言程序:

;从A中取一个字节数据写向I2C总线

WRITE_BYTE: MOV R7,#8 ;写8位

WRITE_LOOP: RLC A ;发送A中数据

MOV SDA,C

SETB SCL

NOP

NOP

NOP

NOP

NOP

CLR SCL

DJNZ R7,WRITE_LOOP

RET

C语言程序:

*/*将指针P指向的一个字节数据发送*/*

void SendByte(uchar *p)

{ uchar n ,temp;

temp =*p;

for(n=0;n<8;n++) */*一字节为8位，循环8次*/*

{if(temp &0x80) SDA=1; */*将数据线SDA置1或清0*/*

else SDA=0;

NOP

SCL=1; */*置SCL为高，通知从机开始接收数据*/*

DELAY5US;

SCL=0; */*SCL变低，准备发送下一位数据*/*

temp = temp <<1; */*准备下一位要发送的数据*/*

}

}

7.从I2C总线接收一个字节 数据汇编语言程序:

; 从I2C总线接收一个字节 数据放在A中

RDBYTE : MOV R7,#8 ;写8位

RD_LOOP: SETB SDA

SETB SCL

NOP; 延时5 μ s

NOP

NOP

NOP

NOP

NOP

MOV C, SDA ;采样SDA线上的数据到cy

MOV A, R2 ;R2为接收数据的缓冲寄存器

RLC A ;将cy中的数据左移进A中

MOV R2, A ;数据送回缓冲寄存器R2

CLR SCL

DJNZ R7,RD_LOOP

RET

C语言程序:

/*接收一个字节数据放在P指向单元*/

uchar RcvByte (uchar *P)

{

uchar n,temp;

for(n=0;n<8;n++) **/*一字节为8位，循环8次*/**

{ SDA=1; **/*置数据线SDA为高,进入接收方式*/**

SCL=1; **/*SCL由低变高，产生一个时钟*/**

DELAY5US;

temp= temp<<1;

if(SDA=1) temp= temp | 0X01

ELSE temp= temp&0xfe;

SCL=0; **/*时钟线SCL清0*/**

}

***p= temp;**

}

8.向有子地址器件发送多个字节

以上介绍的都是I2C总线的基本操作，I2C总线完整的数据传输是由以上操作组合而成。设某从器件的写地址为s1a，如果希望向该器件由子地址suba开始的单元连续写入n个字节的数据dat1、dat2、.....datn，相应的操作过程如图9-12所示。有阴影部分表示数据由主器件向从器件传送，无阴影部分表示数据由从器件向主器件传送。

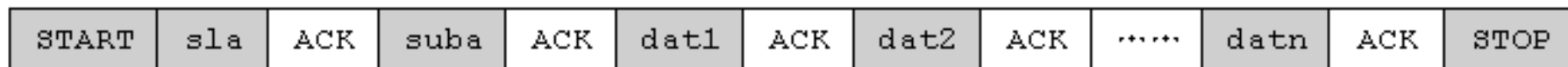


图9-12向有子地址器件发送多个字节的操作过程

汇编语言程序：

；多字节写操作子程序WNBYTE

；入口参数：R7写入的字节数，R0写入数据的首地址，R2从器件地址，
R3从器件内部地址



```
WNBYTE:MOV A,R3;  
        LCALL START  
        LCALL WRITE_BYTE  
        LCALL CACK  
        JB F0,WRBYTE  
        MOV A,R2  
        LCALL WRITE_BYTE  
        LCALL CACK  
        JB F0,WRBYTE  
WRDA: MOV A,@R0  
      LCALL WRITE_BYTE  
      LCALL CACK  
      JB F0,WRBYTE  
      INC R0  
      DJNZ R7,WRDA  
      LCALL STOP  
      RET
```


C语言程序:

/*多字节写操作子程序WNBYTE

入口参数: **n**写入的字节数, **S0**写入数据的首地址, **S3**从器件地址, **S2**从器件地址内部地址*/

void Sendnbyte(uchar *s3,uchar *s2,uchar *s0,uchar n)

```
{ uchar i;
  loop:start( );           /*发起始信号, 启动总线*/
  SendByte(s3);           /*发送从器件地址*/
  cack( );
  if(ack)
goto loop
  SendByte(s2);           /*发送器件子地址*/
  cack( );
  if(ack)
goto loop
  for(i=0;i<n;i++)        /*循环n次*/
  { SendByte(s0);         /*发送一个字节数据*/
    cack( );
    if(ack)
goto loop
    s0++;                 /*指向下一个字节*/
  }
  stop( );               /*发结束信号, 结束本次数据传送 */
}
```

9.向有子地址器件读取多个字节

主机首先发送起始信号、从器件地址s1a和希望读取的字节数据所在从器件子地址suba，执行一次写操作，在从器件应答之后，主器件重新发送起始信号和从器件读地址s1a+1，从器件响应并发送应答信号后，输出主机所要求的一个字节数据dat1，主器件随后发送应答信号ACK，以后从器件每输出一个字节数据，主机均回送ACK应答，当从器件输出最后一个字节数据datn后，主机回送非应答信号，接着发送停止信号结束总线传送，相应的操作过程如图9-13所示。

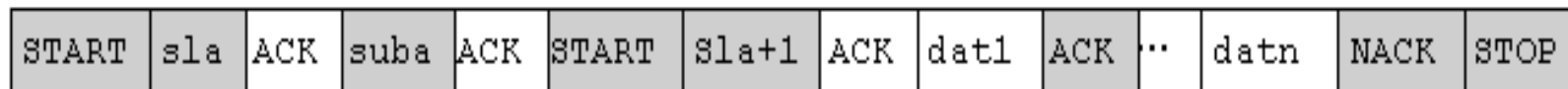


图9-13向有子地址器件读取多个字节的操作过程



汇编语言程序：

； 多字节读操作子程序**RNBYTE**

； 入口参数：**R7**写入的字节数，**R0**写入数据的首地址， **R2**从器件地址内部地址， **R3**从器件写地址， **R4**从器件读地址

RNBYTE: LCALL START

MOV A, R3

；取从器件写地址

LCALL WRITE_BYTE

；写从器件地址

LCALL CACK

；检测应答信号

JB F0, RNBYTE

；无应答重新开始

MOV A, R2

；取从器件地址内部地址

LCALL WRITE_BYTE

LCALL CACK

JB F0, RNBYTE

LCALL START



MOV A, R4 ;取从器件读地址

LCALL WRITE_BYTE

LCALL CACK

JB F0, RNBYTE

RDN: LCALL RDBYTE ;接收一个字节数据

MOV @R0, A

DJNZ R7, ACK

LCALL MNACK ;接收完发非应答信号

LCALL STOP

ACK: LCALL MACK ;没接收完发应答信号

INC R0

SJMP RDN

C语言程序:

/*多字节读操作子程序RNBYTE

/*入口参数: n写入的字节数, s0读数据存放的首地址, s2从器件地址内部地址, s3从器件写地址, s4从器件读地址*/

**viod Rcvnbyte(uchar *s3,uchar *s4, uchar *s2, uchar *s0,
uchar n)**

```
{  
    loop: start( );                                /*发起始信号, 启动总线*/  
        SendByte(s3);                                /*发送从器件地址*/  
        Cack( );                                    /*应答检测*/  
        if(ack)                                       /*如果没能应答, 重新开始 */  
            goto loop  
        SendByte(s2);                                /*发送器件子地址*/  
        Cack( );                                    /*应答检测*/  
        if(ack)                                       /*如果没能应答, 重新开始 */  
            goto loop
```



```
start( );           /*再次发起始信号*/
SendByte(s4);       /*sla+1表示进行读操作*/
cack( );            /*应答检测*/
if(ack)              /*如果没能应答，重新开始 */
goto loop
for(i=0;i<n-1;i++)   /*对前n-1个字节发应答信号*/
{ RcvByte(s0);       /*接收数据*/
  ack( );            /*发送应答信号*/
  s++;
}
RcvByte(s0);         /*接收最后一个字节*/
nack( );             /*发送非应答信号*/
stop( );             /*发结束信号，结束本次数据传送*/
}
```

9.2.3 24CXX系列EEPROM芯片及其与单片机的接口

串行E2PROM是在各种串行器件应用中使用较频繁的器件，和并行E2PROM相比，串行E2PROM容量小、数据传送速度较低，但因其体积较小，引脚较少，功耗低，特别适合于需要存放非挥发数据，速度要求不高，引脚少的单片机应用系统。

24CXX系列的E2PROM有多种型号，其中典型的型号有：

24C02/04/08/16/32/64/128/256共8中芯片，容量分别为1、2、4、8、16、32、64、128、256KB。

串行EEPROM一般具有两种写入方式，一种是字节写入方式，还有另一种页写入方式，允许在一个写周期内同时对1个字节到一页的若干字节的编程写入，一页的大小取决于芯片内页寄存器的大小。其中，24C01具有8字节数据的页面写能力，24C02 / 04 / 08 / 16具有16字节数据的页面写能力，24C32 / 64具有32字节数据的页面写能力，24C128 / 256具有64字节数据的页面写能力。

1、引脚的功能

24CXX系列的E2PROM管脚排列图分别为如图9-14的（a）、（b）、（c）所示

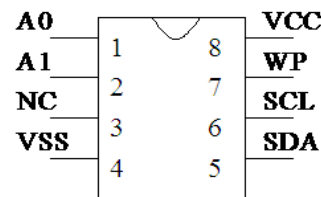
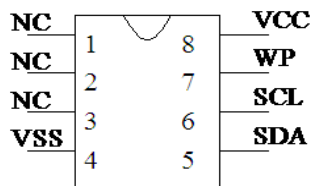
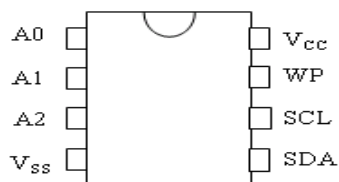
VCC: 电源+5V。

VSS: 地线。

SCL: 串行时钟输入端，用于发送数据或接收数据时产生所需的时钟。

SDA: 串行数据I/O端，用于输入和输出串行数据。该引脚是漏极开路的端口，需接上拉电阻到VCC。

WP: 写保护端，该引脚提供了硬件数据保护，当WP接地时，允许对芯片执行写操作；当WP接VCC时，则对芯片实施写保护。



(a) 24C01/02/04/08/16/32/64管脚 (b) 24C128管脚 (c) 24C256管脚

图9-14 24CXX 系列E2PROM 引脚图



A0、A1、A2：器件地址输入端，用于多个器件级联时设置器件地址，当这些脚悬空时默认值为0，对于24C02可级联8个器件，如果线路上只有一片24C02，这三个地址输入脚A0、A1、A2 可悬空或连接到GND。

例：如果A2、A1、A0所接的电平为101，由于24C02的器件标识为1010，那么，该芯片的读地址为0xab, 写地址为0xaa。

2. 24CXX的器件地址

24CXX的器件地址见表9-6所示。

表9-6 24CXX的器件地址

型号	控制码	片选	读写	总线访问的器件
24C01	1010	A2 A1 A0	1/0	最多8个
24C02	1010	A2 A1 A0	1/0	最多8个
24C04	1010	A2 A1 a8	1/0	最多4个
24C08	1010	A2 a9 a8	1/0	最多2个
24C16	1010	a10 a9 a8	1/0	最多1个
24C32	1010	A2 A1 A0	1/0	最多8个
24C64	1010	A2 A1 A0	1/0	最多8个
24C128	1010	X X X	1/0	最多1个
24C256	1010	0 A1 A0	1/0	最多4个

3. AT89S51单片机与24C02的硬件连接

图9-15为单片机与24C02的连接，其中P1.0作为24C02的数据线SDA，P1.1作为24C02的时钟线SCL，24C02的器件标识地址为1010。

由于系统中只有一片24C02，所以直接将器件地址输入端A2、A1、A0接地，这样24C02在系统中的器件地址SLAW=0xA0，SLAR=0xA1。两条线均接4.7K的上拉电阻

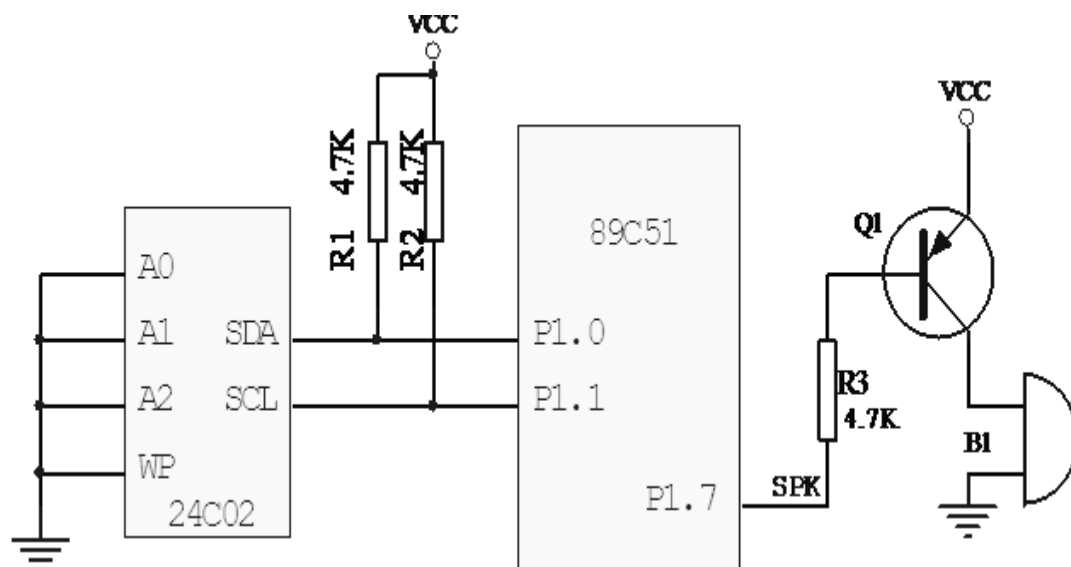


图9-15 单片机与24C02的连接电路

4. 89S51对24C02的读写程序

针对图9-15，编写对24C02的读写程序，将若干个字节数据写到24C02芯片地址为0x10开始的单元中，随后从这些单元读出数据，判断是否与写入的数据一致，如果读/写正确，蜂鸣器鸣叫一声，否则鸣叫3声。

24C02的内部有连续的子地址空间，对这些空间进行n个字节的连续读/写时，具有地址自动加1功能，只要设定好希望读/写的器件子地址及字节数，就能完成整个操作。
注意：对于24C02连续写的字节数不应超过页容量8，一次连续写所形成的总线传送结束后（主机发出停止信号后），24C02执行内部擦写过程，大约需要10ms左右，此时24C02不再应答主器件的任何请求。



C语言参考程序见课本.

其中的函数Sendnbyte()、Rcvnbyte()在前面9.2.2节已作介绍,对24C02的读写完全适用。为了方便,可以将9.2.2节介绍的模拟I2C总线操作的子程序形成一个文件I2C.C,并形成如下的I2C.h 文件,主程序和I2C.C加入同一个项目中,并且主程序包含这个h文件,即可使用Rcvnbyte()、Rcvnbyte ()这两个函数。

9.2.4 数码管动态显示驱动、键盘扫描管理芯片 ZLG7290B及与单片机接口

ZLG7290B 是广州周立功单片机发展有限公司自行设计的数码管显示驱动及键盘扫描管理芯片。能够直接驱动8位共阴式数码管（或64只独立的LED），同时还可以扫描管理多达64只按键。其中有8只按键还可以作为功能键使用，就像电脑键盘上的Ctrl、Shift、Alt键一样。另外ZLG7290B 内部还设置有连击计数器，能够使某键按下后不松手而连续有效。采用I2C 总线方式，与微控制器的接口仅需两根信号线。该芯片为工业级芯片，抗干扰能力强，在工业测控中已有大量应用。

1. 主要特性

- 直接驱动8位共阴式数码管或64只独立的LED；

- 能够管理多达64只按键，自动消除抖动，其中有8只可以作为功能键使用；

- 段电流可达 20mA，位电流可达100mA以上；

- 利用功率电路可以方便地驱动1英寸以上的大型数码管；

具有闪烁、段点亮、段熄灭、功能键、连击键计数等强大功能；
提供有10种数字和21种字母的译码显示功能，或者直接向显示缓存写入显示数据；
不接数码管而仅使用键盘管理功能时，工作电流可降至 1mA；
与微控制器之间采用I2C串行总线接口，只需两根信号线，节省I/O资源；
工作电压范围：+3.3~5.5V；
工作温度范围：-40~+85℃；
封装：DIP-24（窄体），SOP-24。

2. 引脚图及功能说明

引脚图见9-16所示，引脚功能见表9-7所示。

1	SC/KR2	KR1/SB	24
2	SD/KR3	KR0/SA	23
3	DIG3/KC3	KC4/DIG4	22
4	DIG2/KC2	KC5/DIG5	21
5	DIG1/KC1	SDA	20
6	DIG0/KC0	SCL	19
7	SE/KR4	OSC2	18
8	SF/KR5	OSC1	17
9	SG/KR6	VCC	16
10	DP/KR7	RST	15
11	GND	INT	14
12	DIG6/KC6	KC7/DIG7	13

图 9-16 ZLG7290B 引脚图（DIP-24，SOP-24）

表 9-7 ZLG7290B 引脚功能表

引脚序号	引脚名称	功能描述
1	SC/KR2	数码管c段/键盘行信号2
2	SD/KR3	数码管d段/键盘行信号3
3	DIG3/KC3	数码管位选信号3/键盘列信号3
4	DIG2/KC2	数码管位选信号2/键盘列信号2
5	DIG1/KC1	数码管位选信号1/键盘列信号1
6	DIG0/KC0	数码管位选信号0/键盘列信号0
7	SE/KR4	数码管e段/键盘行信号4
8	SF/KR5	数码管f段/键盘行信号5
9	SG/KR6	数码管g段/键盘行信号6
10	DP/KR7	数码管dp段/键盘行信号7
11	GND	接地
12	DIG6/KC6	数码管位选信号6/键盘列信号6

引脚序号	引脚名称	功能描述
13	DIG7/KC7	数码管位选信号7/键盘列信号7
14	/INT	键盘中断请求信号，低电平（下降沿）有效
15	/RST	复位信号，低电平有效
16	Vcc	电源，+3.3~5.5V
17	OSC1	晶振输入信号
18	OSC2	晶振输出信号
19	SCL	I ² C总线时钟信号
20	SDA	I ² C总线数据信号
21	DIG5/KC5	数码管位选信号5/键盘列信号5
22	DIG4/KC4	数码管位选信号4/键盘列信号4
23	SA/KR0	数码管a段/键盘行信号0
24	SB/KR1	数码管b段/键盘行信号1

3. ZLG7290B 典型应用电路原理图如图9-17所示。

(1) 电路简析

在图9-17中，U1就是ZLG7290B。J1 是ZLG7290B与微控制器的接口，按照I2C 总线协议的要求，信号线SCL和SDA上必须要分别加上上拉电阻，其典型值是10K Ω 。晶振Y1通常取值4MHz，调节电容C3和C4通常取值在10pF左右。复位信号是低电平有效，数码管必须是共阴式的，不能直接使用共阳式的。DPY1和DPY2是4位联体式数码管，共同组成完整的8位。R1~R8是限流电阻，典型值是270 Ω 。64只按键中，前56个按键是普通按键K1~K56，最后8个为功能键F0~F7。数码管扫描线和键盘扫描线是共用的，所以二极管D1~D8 是必须的，有了它们就可以防止按键干扰数码管显示的情况发生。在多数应用当中可能不需要太多的按键，这时可以按行或按列裁减键盘。

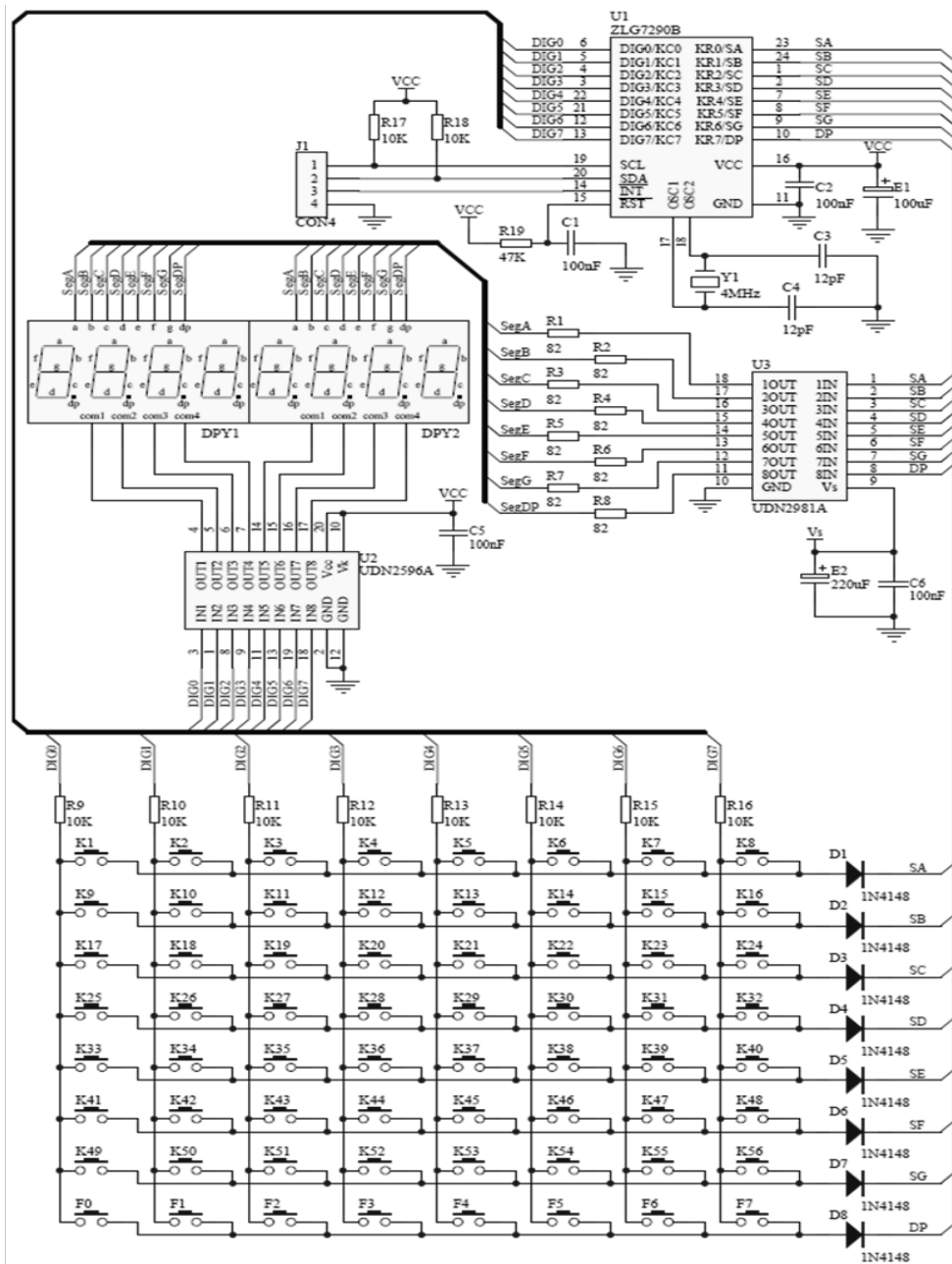


图 9-17 ZLG7290B 典型应用电路原理图

（2）功能概述

如图9-17所示，ZLG7290B可以扫描管理多达64个按键，K1~K56 为普通按键，F0~F7 为功能键。普通按键还有连击检测功能。ZLG7290B 内部有8 个显示缓冲寄存器DpRam0~DpRam7，它们直接决定数码管显示的内容。

ZLG7290B 提供有两种显示控制方式，一种是直接向显存写入字型数据，另一种是通过向命令缓冲寄存器写入控制指令实现自动译码显示。访问这些寄存器需要通过 I2C 总线接口来实现。ZLG7290B 的I2C 总线器件地址是70H（写操作）和71H（读操作）。访问内部寄存器要通过“子地址”来实现。

4. 寄存器详解

（1）系统寄存器SystemReg（地址：00H）

系统寄存器的第0位称作KeyAvi，标志着按键是否有效，0—没有按键被按下，1—有某个按键被按下。SystemReg 寄存器的其它位暂时没有定义。当按下某个键时，7290B 的INT 引脚会产生一个低电平的中断请求信号。当读走键值后，中断信号就会自动撤销。

（2）键值寄存器Key（地址：01H）

如果某个普通键（图 9-23中的K1~K56）被按下，则微控制器可以从键值寄存器Key中读取相应的键值1~56。如果微控制器发现ZLG7290B的INT引脚产生了中断请求，而从Key中读到的键值是0，则表示按下的可能是功能键。键值寄存器Key的值在被读走后自动变成0。

3) 连击计数器RepeatCnt (地址: 02H)

ZLG7290B 为普通键 (图9-中的K1~K56) 提供了连击计数功能。所谓连击是指按住某个普通键不松手, 经过一两秒钟的延迟后, 开始连续有效, 连续有效间隔时间为几十到几百个毫秒。当按住某个普通键一直不松手时: 首先会产生一次中断信号, 这时连击计数器RepeatCnt的值仍然是0; 经过一两秒延迟后, 会连续产生中断信号, 每中断一次RepeatCnt 就自动加1; 当RepeatCnt计数到255时就不再增加, 而中断信号继续有效。

（4） 功能键寄存器FunctionKey（地址：03H）

ZLG7290B 还提供有8个功能键（图9-23中的F0～F7）。功能键常常是配合普通键一起使用的，就像电脑键盘上的Shift、Ctrl和Alt键，当然也可以单独使用。当按下某个功能键时，在/INT 引脚也会像按普通键那样产生中断信号。功能键的键值是被保存在unctionKey 寄存器中的。功能键寄存器FunctionKey 的初始值是FFH，每一个位对应一个功能键，第0 位（LSB）对应F0，第1位对应F1，依次类推，第7位（MSB）对应F7。某一功能键被按下时，相应的FunctionKey 位就清零。

（5 ）命令缓冲区CmdBuf0 和CmdBuf1（地址：07H 和08H）

通过向命令缓冲区写入相关的控制命令可以实现段寻址、下载显示数据、控制闪烁等功能。

（6）闪烁控制寄存器Flash On Off（地址：0CH）

Flash On Off寄存器决定闪烁频率和占空比。复位值为01110111B。高4位表示闪烁时亮的持续时间，低4位表示闪烁时灭的持续时间。改变Flash On Off 的值，可以同时改变闪烁频率和占空比。特别说明：单独设置Flash On Off寄存器的值，并不会看到显示闪烁，而应该配合闪烁控制命令一起使用。

（7）扫描位数寄存器ScanNum（地址：0DH）

ScanNum 寄存器决定扫描显示的位数，取值0~7，对应1~8 位。复位值是7，即数码管的8 位都扫描显示。实际应用中可能需要显示的位数不足8位，例如只显示3位，这时可以把ScanNum 的值设置为2，则数码管的第0、1、2 位被扫描显示，而第3~7 位不会被分配扫描时间，所以不显示。数码管的扫描位数减少后，有用的显示位由于分配的扫描时间更多因而显示亮度得以提高。ScanNum 寄存器的值为0时，只有数码管的第0位在显示，亮度达到最大。

(8) 显示缓冲区DpRam0~DpRam7 (地址: 10H~17H)

DpRam0~DpRam7 这8 个寄存器的取值直接决定了数码管的显示内容。每个寄存器的8 个位分别对应数码管的 a, b, c, d, e, f, dp 段, MSB 对应a, LSB 对应dp。例如大写字母H 的字型数据为6EH (不带小数点) 或6FH (带小数点)。

5. 控制命令详解

寄存器 CmdBuf0（地址：07H）和CmdBuf1（地址：08H）共同组成命令缓冲区。通过向命令缓冲区写入相关的控制命令可以实现段寻址、下载显示数据、控制闪烁等功能。

（1）段寻址命令（Seg On Off）

段寻址命令格式见表9-8所示。

表9-8段寻址命令格式

D7	D6	D5	D4	D3	D2	D1	D0		D7	D6	D5	D4	D3	D2	D1	D0
0	0	0	0	0	0	0	1		on	0	S ₅	S ₄	S ₃	S ₂	S ₁	S ₀

在段寻址命令中，8位数码管被看成是64个段，每一个段实际上就是一只独立的LED。第1字节00000001B 是命令字，为该命令的特征码；第二字节中的on 表示该段是否点亮，0—灭，1—亮；S₅S₄S₃S₂S₁S₀ 是64位段地址，取值0～63。在某1位数码管内，各段的亮或灭的顺序按照a, b, c, d, e, f, g, dp 进行。

(2) 下载数据命令 (Down load)

下载数据命令格式见表9-9所示。

表9-9下载数据命令

D7	D6	D5	D4	D3	D2	D1	D0		D7	D6	D5	D4	D3	D2	D1	D0
0	1	1	0	A ₃	A ₂	A ₁	A ₀		dp	flash	0	d ₄	d ₃	d ₂	d ₁	d ₀

在第一字节中，高4位的0110 是命令特征码；A₃A₂A₁A₀ 是数码管显示数据的位地址（其中A₃ 留作以后扩展之用，实际使用时取0 即可），位地址编号按从左到右的顺序依次为0, 1, 2, 3, 4, 5, 6, 7（以第本章中的图9-23为准）；dp 控制小数点是否点亮，0—点亮，1—熄灭；flash 表示是否要闪烁，0—正常显示，1—闪烁；d₄d₃d₂d₁d₀ 是要显示的数据，包括10 种数字和21 种字母。显示数据按照表9-10中的规则进行译码。

表9-10 下载数据并译码命令的数据表

d ₄ d ₃ d ₂ d ₁ d ₀ (二进制)					d ₄ d ₃ d ₂ d ₁ d ₀ (十进制)	显示结果
0	0	0	0	0	00H	0
0	0	0	0	1	01H	1
0	0	0	1	0	02H	2
0	0	0	1	1	03H	3
0	0	1	0	0	04H	4
0	0	1	0	1	05H	5
0	0	1	1	0	06H	6
0	0	1	1	1	07H	7
0	1	0	0	0	08H	8

表9-10 下载数据并译码命令的数据表

d ₄ d ₃ d ₂ d ₁ d ₀ (二进制)					d ₄ d ₃ d ₂ d ₁ d ₀ (十进制)	显示结果
0	1	0	0	1	09H	9
0	1	0	1	0	0AH	A
0	1	0	1	1	0BH	b
0	1	1	0	0	0CH	C
0	1	1	0	1	0DH	d
0	1	1	1	0	0EH	E
0	1	1	1	1	0FH	F
1	0	0	0	0	10H	G
1	0	0	0	1	11H	H

表9-10 下载数据并译码命令的数据表

$d_4d_3d_2d_1d_0$ (二进制)					$d_4d_3d_2d_1d_0$ (十进制)	显示结果
1	0	0	1	0	12H	i
1	0	0	1	1	13H	j
1	0	1	0	0	14H	L
1	0	1	0	1	15H	o
1	0	1	1	0	16H	p
1	0	1	1	1	17H	q
1	1	0	0	0	18H	r
1	1	0	0	1	19H	t

表9-10 下载数据并译码命令的数据表

d ₄ d ₃ d ₂ d ₁ d ₀ (二进制)					d ₄ d ₃ d ₂ d ₁ d ₀ (十进制)	显示结果
1	1	0	1	0	1AH	U
1	1	0	1	1	1BH	y
1	1	1	0	0	1CH	c
1	1	1	0	1	1DH	h
1	1	1	1	0	1EH	T
1	1	1	1	1	1FH	(无显示)

(3) 闪烁控制 (Flash)

闪烁控制命令格式见表9-11所示。

表9-11 闪烁控制

D7	D6	D5	D4	D3	D2	D1	D0		D7	D6	D5	D4	D3	D2	D1	D0
0	1	1	1	X	X	X	X		F ₇	F ₆	F ₅	F ₄	F ₃	F ₂	F ₁	F ₀

在命令格式中，高4 位的0111 是命令特征码；xxxx 表示无关位，通常取值0000；第2 字节的Fn（n=0~7）控制数码管相应位的闪烁属性，0—正常显示，1—闪烁。复位后，所有位都不闪烁。

6. ZLG7290动态数码显示程序设计

在8个数码管上从左到右显示” 20100725”,参考程序清单如下:

```
SDA  BIT  P1.0
SCL  BIT  P1.1
WSLA EQU 070H; ZLG7290器件的写地址
RSLA EQU 071H; ZLG7290器件的读地址
ORG      0000H
LJMP 0100H
ORG 0100H      ;主程序
START:      MOV  30H,#02H      ;变量缓冲区定义显示
“20100725”
MOV  31H,#00H      ;
MOV  32H,#01H      ;
MOV  33H,#00H
MOV  34H,#00H
```



```
MOV 35H,#07H
MOV 36H,#02H
MOV 37H,#05H
MOV DPTR,#LEDSEG ;数据指针指向字型码
```

表首地址

```
CLR A
MOV R7,#08H
MOV R0,#40H
MOV R1,#30H
```

```
LOOP1: MOVA, @R1 ;从变量缓冲区取出要形式的
```

数字

```
MOVC A,@A+DPTR ;查表得字型
```

码

```
MOV @R0,A ;将字型码存储到40H开始的单
```

元中

```
INC R1
INC R0
DJNZ R7,LOOP1
```



LOOP: MOV R7,#08H

MOV R0,#40H ;字型码首地址送R0

MOV R2,#10H ;ZLG7290内部显示缓冲区首地址送R2

MOV R3,#WSLA ;ZLG7290器件的写地址送R3

LCALL WRNBYT ; 调用显示子程序

LCALL DELAY

SJMP LOOP

LEDSEG: DB 0FCH,60H,0DAH,0F2H,66H,0B8H,0BEH,0E4H ;0-F共

阴字型码表

DB 0FEH,0F6H,0EEH,3EH,9CH,7AH,9EH,8EH

DELAY: MOV R5,#00H ;延时子程序

DELAY1: MOV R6,#00H

DJNZ R6,\$

DJNZ R5,DELAY1

RET



上述调用的相关的I2C子程序

(WRNBYT,WRBYT,STOP,CACK,START)参见9.2.2节的内容。

采用C语言编写的参考程序

```
#include "reg51.h"
#include "intrins.h"
#define DEPLAY5US _nop_(); _nop_(); _nop_();
_nop_(); _nop_();
sbit SDA=P1^0;
sbit SCL=P1^1;
#define WSLA0 0x70
#define RSLA0 0x71
#define uchar unsigned char
void STA(void);
void STOP(void);
void CACK(void);
```

```
void Sendbyte (unsigned char *p);
void Sendnbyte(uchar *s3,uchar *s2,uchar *s0,uchar n);
void DELAY();
void main()
{
    uchar n,i,m,*c,*y,*x,wsubsla=0x10,WSLA=WSLA0;
    uchar a[8]={2, 0, 1, 0, 0, 7, 2, 5};    /*显示字符*/
    uchar b[8];                               /*存放显示字符对应的字型码*/
    uchar
    zxm[16]={0xfc,0x60,0xda,0xf2,0x60,0xda,0xf2,0x66,0xbe,0xe4,
    0xfe,0xf6,0xee,0x3e,0x9c,0x7a, 0x9e,0x8e};    /*0-F的字型码 */
    for(i=0;i<8;i++)
    {
        m=a[i];                               /*取当前显示字符*/
        b[i]=zxm[m]                            /*查得显示字符的字型码 */
    }
}
```



```
while(1)
```

```
{
```

```
    x=&WSLA;
```

```
    /* 取ZLG7290器件的写地址*/
```

```
    c=&wsubsla;
```

```
    /* 取ZLG7290器件的内部显示缓冲
```

```
寄存器的地址*/
```

```
    y=b;
```

```
    /*获得显示字符的字型码地址 */
```

```
    n=8;
```

```
    Sendnbyte (x,c,y,n);
```

```
    /*调用写多字节的显示子程序*/
```

```
    DELAY( );
```

```
}
```

```
}
```

```
void DELAY( )
```

```
{
```

```
    unsigned char i,j;
```

```
    for(i=0;i<100;i++)
```

```
        for(j=0;j<100;j++);
```

```
}
```

7. 键盘扫描程序设计

键盘扫描程序，将得到的键值（01H-10H）在最右边两位数码管显示，显示的格式为：“data=xx”，程序采用中断结构，硬件连接上将INT_KEY信号与P3.2(INT0)连接；普通的I2C通讯程序可以直接利用，ZLG7290芯片在读数据时有20 μ s延时

汇编语言程序：

```
SDA    BIT    P1.0
SCL     BIT    P1.1
WSLA   EQU    070H ; ZLG7290器件的写地址
RSLA   EQU    071H ; ZLG7290器件的读地址
ORG     0000H
LJMP    0100H
ORG     0003H      ;中断入口
LJMP    INT_7290   ;键盘中断服务
ORG     0100H
```



START:MOV SP,#60H

SETB EA ;开INT0中断

SETB EX0

SETB IT0 ;触发极性为下降沿

MOV 30H, #0DH ;变量缓冲区（存放显示字符在字型码表中的偏移地址）

MOV 31H, #10H

MOV 32H, #11H

MOV 33H, #10H

MOV 34H, #02H

MOV 35H, #13H ;

MOV 36H, #13H ;

MOV 37H, #13H ;

;通过查表建立显示缓冲区（40H-47H）



```
MOV DPTR,#LEDSEG      ;开始对变量查表
MOV R7,#8              ;写入数据个数
MOV R0,#30H            ;源数据块首地址
MOV R1,#40H            ;当前字符字形码表显示缓冲
```

区

```
LOOP1:MOV      A,@R0
      MOVC  A,@A+DPTR      ;查表得对应的字形码
      MOV   @R1,A          ;送显示缓冲区
      INC   R1
      INC   R0
      DJNZ  R7,LOOP1
;向7290B写入数据，以显示“data= XX”，最后两位为键值的十进制显示
```

LOOP: MOV R7,#8

MOV R2,#10H
冲寄存器的地址

MOV R3,#WSLA
R2

MOV R0,#40H
区地址送R0

LCALL WRNBYT

LCALL DELAY

SJMP LOOP

LEDSEG:DB 0FCH,60H,0DAH,0F2H,66H,0B6H,0BEH,0E4H

;0-7的字形码

DB 0FEH,0F6H,0EEH,3EH,9CH,7AH,9EH,8EH

;8-F的字形码

DB 0FAH,1EH,12H,00H

;a,t,=和熄灭码

CF: PUSH 02H

制数并查表

;ZLG7290器件的内部显示缓

; ZLG7290器件的写地址送

;当前字符字形码表显示缓冲

;调显示子程序

;使显示稳定

;将A中的数据拆分为两个四位16进



```
PUSH DPH ;
PUSH DPL
MOV DPTR,#LEDSEG
MOV R2,A
ANL A,#0FH
MOVC      A,@A+DPTR
MOV R3,A
MOV A,R2
SWAPA
ANL A,#0FH
MOVC      A,@A+DPTR
MOV R4,A
POP DPL
POP DPH
POP 02H
RET
```

**;中断服务程序INT_7290:(读取健值、拆分并转换成
字型码更新46H和47H单元内容，以便刷新显示)**

INT_7290:NOP

PUSH 00H

PUSH 02H

PUSH 03H

PUSH 04H

PUSH 07H

PUSH ACC

PUSH PSW

MOV R0,#20H

;状态数据区首址

MOV R7,#04H

;取状态数据个数

MOV R2,#00H

;内部数据首地址



MOV R3,#WSLA ;取器件地址（写）
MOV R4,#RSLA ;取器件地址（读）
LCALL RNBYTE ;读出7290的4个寄存器（地
址为00H—03H）数据存于20H-23H
NOP
MOV A,21H ;取21H单元的键值
LCALL CF ;拆分、查表
MOV 47H,R3 ;送显示缓冲区（最低两位数码
管的字型码在46H、47H中）
MOV 46H,R4
POP PSW
POP ACC
POP 07H
POP 04H



```
POP 03H
```

```
POP 02H
```

```
POP 00H
```

```
RETI
```

```
DELAY: MOV R6,#00H
```

```
DELAY 1: MOV R5,#00H
```

```
DJNZ R5,$
```

```
DJNZ R6,DELAY1
```

```
RET
```

```
END
```

相关的I2C子程序（WRNBYT、RNBYTE、WRBYT、STOP、CACK、STA），参见9.2.2的内容。



采用C语言编写的参考程序

```
#include "reg51.h"
```

```
#include "intrins.h"
```

```
#define DELAY5US
```

```
_nop();_nop();_nop();_nop();_nop();
```

```
sbit SDA=P1^0;
```

```
sbit SCL=P1^1;
```

```
#define unsigned char uchar
```

```
#define WSLA1 0x70;
```

/*7290器件地址*/

```
#define RSLA1 0x71;
```

```
void start(void);
```

/*函数声明*/

```
void stop(void);
```

```
void mack(void);
```



```
void nmack(void);
void cack(void);
void Sendbyte (unsigned char *p);
void Rcvbyte (unsigned char *p);
void Sendnbyte (unsigned char *s3, unsigned
char *s2, unsigned char *s0, unsigned char n);
void Rcvnbyte (unsigned char *s3, unsigned
char *s4, unsigned char *s2, unsigned char
*s0, unsigned char n);
uchar  zxm[8];
uchar  code
b[20]={0xfc,0x60,0xda,0xf2,0x66,0xb6,0xde,0xe
4,0xfe,0xf6,0xee,
0x3e,0x9c,0x7a,0x9e,0x8e,0xfa,0x1e,0x12,0x00};
/*0-F字型码，最后4个依次为a,t,=和熄灭的字型码*/
```



```
void DELAY();
void main()
{
    uchar n,i,*c,*y,*x,wsubsa=0x10,WSLA=WSLA1;
    uchar a[8]={ 0x0d, 0x10, 0x11, 0x10,0x12,0x13,0x13,0x13};/*变量
缓冲区（存放显示字符在字型码表中的偏移地址）*/
    for(i=0;i<8;i++)
        zxm[i]=b[a[i]];          /*将显示字符查表转换成字型码*/
    EA=1;                        /*中断初始化*/
    EX0=1;
    IT0=1;
    while(1)
    {
        x=&WSLA;                /* 取ZLG7290器件的写地址*/
        c=&wsubsa;              /* 取ZLG7290器件的内部显示缓冲寄存器
的地址*/
        y=zxm;                  /*取显示字符字型码首地址*/
        n=8;
        Sendnbyte (x,c,y,n);    /* 调用写多字节的显示子程序*/
        DELAY();
    }
}
```

```
void INT_7290() interrupt 0 using 0  /*7290的中断函数，读取健值、拆分  
并转换成字型码更新数组zxm[6]和zxm[7],以便刷新显示*/  
{  
    uchar  
n=4,i,dyuan[4],*c,*y,*x,*d,wai=0x00,WSLA=WSLA1,RSSLA=RSLA1;  
    y=dyuan;  
    c=&wai;  
    x=&WSLA;                /* 取ZLG7290器件的写地址*/  
    d=&RSLA;                /* 取ZLG7290器件的读地址*/  
    Rcvnbyte (x,d,c,y,n);    /*读出7290的4个寄存器内容存于数组dyuan[]  
中*/  
    i=dyuan[1];    /*取健值拆分成两位BCD码并分别查表获得字型码在最抵  
两位显示*/  
    i=i&0x0f;  
    zxm[7]=b[i];  
    i=dyuan[1]>>4;  
    i=i&0x0f;  
    zxm[6]=b[i];  
}
```



```
void DELAY()
{
    unsigned char i,j;
    for(i=0;i<100;i++)
    for(j=0;j<100;++);
}
```

相关的I2C子程序（Sendnbyte ()、Rcvnbyte ()、Sendbyte ()、Rcvbyte ()、stop ()、cack ()、start ()），参见9.2.2的内容。

9.3 单总线(1-wire)接口

9.3.1 1-wire简介

单总线(1-wire)是美国DALLAS公司的一项专有技术,它使用一根导线对信号进行双向传输,具有接口简单、容易扩展等优点,适合单主机、多从机构成的系统。

9.3.2 DS18B20简介

DS18B20是美国DALLAS公司推出的数字温度传感器,将温度传感器、数字转换电路集成到了一起,外形如同一只三极管。

1. DS18B20的主要特性

(1) 适应电压范围宽: 3.0~5.5V, 在寄生电源方式下可由数据线供电。

(2) 独特的单线接口方式: DS18B20与微处理器连接时仅需要一条信号线即可实现微处理器与DS18B20的双向通信。

(3) 测温范围: $-55^{\circ}\text{C} \sim +125^{\circ}\text{C}$, 在 $-10^{\circ}\text{C} \sim +85^{\circ}\text{C}$ 时精度为 $\pm 0.5^{\circ}\text{C}$ 。

(4) 编程可实现分辨率: 为9~12位, 对应的可分辨温度分别为 0.5°C 、 0.25°C 、 0.125°C 和 0.0625°C , 可实现高精度测温。

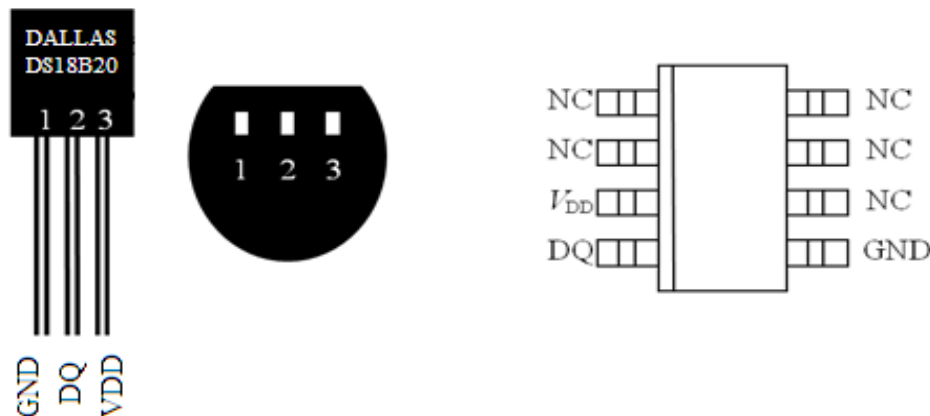
(5) 在9位分辨率时: 最多在93.75ms内把温度值转换为数字,
12位分辨率时: 最多在750ms内把温度值转换为数字。

(6) 支持多点组网功能, 多个DS18B20可以并联在一条DQ线上, 实现“一线制”单主机-多从机分布式温度采集系统, DS18B20依靠各自的序列号采用分时方式与主控器“点对点”通信。

(7) 用户可自设定非易失性的报警上下限值。

2. DS18B20的引脚定义及封装

DS18B20有塑封（TO-92）和扁平封装（8-PIN SOIC）两种形式，如图9-18所示。



(a) 塑封（TO-92）

(b) 扁平封装（8-PIN SOIC）

图9-18 DS18B20的封装和引脚图

引脚功能定义如下：

DQ：数据输入输出，可直接与单片机的I/O口相连。

VDD：+5V电源电压。

GND：电源地。

3. DS18B20的序列号

每片DS18B20均有一个唯一产品序列号，固化在内部的64位激光ROM中，其格式如图9-19所示。开始8位是产品的类型编号（工厂代码）；接着是每个器件唯一的序号，共48位；最后8位是针对前面56位的CRC校验码；这也是多个DS18B20可以采用一条数据线进行通信的原因，只要单片机用匹配命令即可通过序列号访问总线上指定的DS18B20。

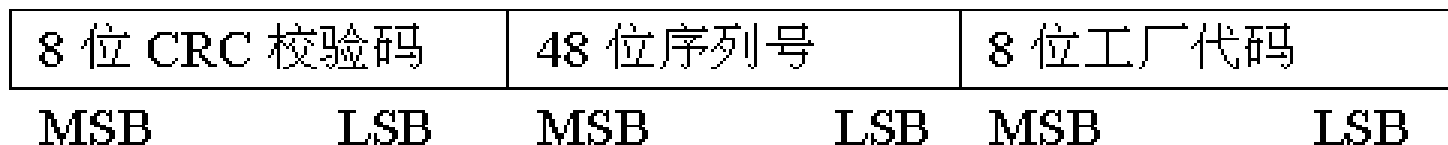


图9-19 64位激光ROM格式

4. 温度暂存器

DS18B20内部有 9个字节的暂存器，见表9-12所示。

表9-12 DS18B20内部有 9个字节的暂存器功能

字节序号	功 能
0	温度转换后的低字节
1	温度转换后的高字节
2	高温触发器TH
3	低温触发器TL
4	配置寄存器
5	保留
6	保留
7	保留
8	CRC校验寄存器

开始两个暂存器（TMSB、TLSB）存放当前测到的温度值，以16位补码形式表示12位温度读数，分辨率为1/16℃。表9-13 是16位温度转换值的存放格式，高5位是温度值的符号扩展，中间7位是温度值的整数部分，最低4位为小数部分。如果测得的温度大于0，高5位为0，只要将测到的数值乘以0.0625即可得到实际温度；如果温度小于0，高5位为1，测到的数值需要取反加1再乘以0.0625即可得到实际温度。温度与转换后的数字量的对应关系见表9-14所示。

表9-13 DS18B20 16位温度转换值格式

MS Byte								LS Byte							
D15	D14	D13	D12	D11	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0
S	S	S	S	S	2 ⁶	2 ⁵	2 ⁴	2 ³	2 ²	2 ¹	2 ⁰	2 ⁻¹	2 ⁻²	2 ⁻³	2 ⁻⁴

表9-14温度与转换后的数字量的对应关系

度/℃	16位二进制编码	十六进制表示
+125	0000 0111 1101 0000	07D0H
+85	0000 0101 0101 0000	0550H
+25. 0625	0000 0001 1001 0001	0191H
+10. 125	0000 0000 1010 0010	00A2H
+0. 5	0000 0000 0000 1000	0008H
0	0000 0000 0000 0000	0000H
-0. 5	1111 1111 1111 1000	FFF8H
-10. 125	1111 1111 0101 1110	FF5EH
-25. 0625	1111 1110 0110 1111	FE6FH
-55	1111 1100 1001 0000	FC90H

高温度触发器和低温度触发器分别存放温度报警的上限值TH和下限值TL。DS18B20完成温度转换后就把转换后的温度值t和报警的上限值TH和下限值TL进行比较，如果 $t > TH$ 或 $t < TL$ ，则把该器件的告警标志置位，并对主机发出的告警搜索命令作出响应。配置寄存器用于确定温度值的数字转换分辨率，该字节各位的意义见表9-15所示。低5位均为1，TM是测试模式位，出厂时设为0，工作模式，不要改变。R0、R1用来设置分辨率，具体见表9-16。出厂时设为12位方式。

表9-15配置寄存器格式

D7	D6	D5	D4	D3	D2	D1	D0
TM	R1	R0	1	1	1	1	1



表9-16 温度分辨率设置

R1	R0	分辨率/位	温度最大转换时间/ms
0	0	9	93.75
0	1	10	187.5
1	0	11	275.00
1	1	12	750.00

9.3.3 DS18B20的读写时序

DS18B20与单片机之间的数据传送是靠严格的时序来实现的，下面来介绍它的初始化及读写时序。

1. 初始化时序

与DS18B20的通信前，首先必图9-20 DS18B20初始化时序须对其初始化。其初始化时序见图9-20。

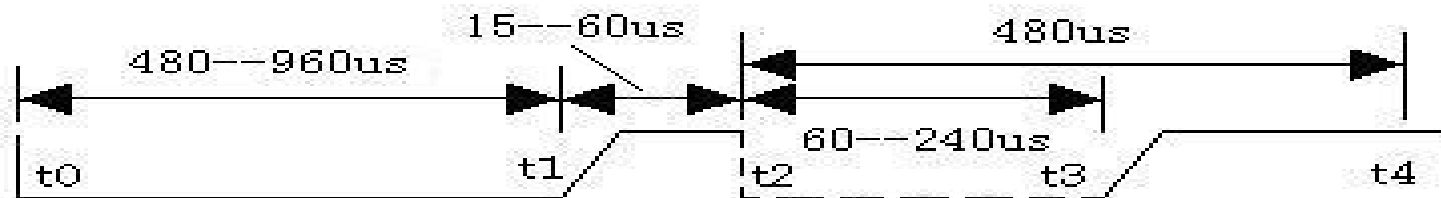


图9-20 DS18B20初始化时序

单片机在t0时刻发出最短为480μs的低电平有效的复位脉冲。在t1时刻释放总线并进入接收状态，DS18B20检测到总线变高后，等15到60μs，在t2时刻发出低电平有效的存在脉冲响应。假设DS18B20和单片机的P1.0引脚相连，晶振12MHZ。初始化程序如下：



汇编语言程序：

DQ BIT P1.0

...

DELAY500US: MOV R7, #00H

DJNZ R7, \$

RET

DELAY60US: MOV R7, #1EH

DJNZ R7, \$

RET

DS18B20_INIT: SETB DQ

NOP

NOP

CLR DQ

；在数据线上产生

500us的低电平



INIT_da18b20:CALL DELAY500US

SETB DQ ; 数据线拉高，进入接收
状态，延时**60us**

CALL DELAY60US

MOV C,DQ ; 检测存在脉冲

JC ERROR ; 没有存在脉冲响应转

CALL DELAY60US ; 检测到存在脉冲响应延

时**240us**

CALL DELAY60US

CALL DELAY60US

CALL DELAY60US

RET

ERROR:CLR DQ

SJMP DS18B20_INIT

RET

C语言程序:

```
sbit DQ=P1^0;
```

```
...
```

```
void delay(unsigned int i)    /*延时10us左右*/
```

```
{ unsigned int j;
```

```
    for (j=i;j > 0;j--);
```

```
}
```

```
bit reset(void)                /*复位函数, 返回值为0复位成功; f返  
回值为1复位失败*/
```

```
{
```

```
    bit flag;
```

```
    DQ=0;                        /*在数据线上产生500us的低电平 */
```

```
    delay(50);
```

```
    DQ=1;                        /*数据线拉高 , 延时30us*/
```

```
    delay(3);
```



```
flag=DQ;          /*读取数据线状态flag =0:复位成功;
flag =1:复位失败*/
delay(25);        /*延时250us*/
return(flag);
}
void ds18b20_init(void)
{ while(1)
    {
        if(!reset ())          /*收到ds18b20的应答信号*/
        {
            DQ=1;
            delay(30);          /*延时300 us */
            break;
        }
        else
            reset();            /*否则再发复位信号*/
    }
}
```

2. 写时序

DS18B20写时序如图9-21，单片机在 t_0 时刻将总线拉至低电平，从 t_0 时刻开始的 $15\mu\text{s}$ 之内应将要写的数据位送到总线上。在 t_0 后的 $15\text{—}60\mu\text{s}$ 内对总线采样，若为低电平，写入的是0，连续写2位之间的间隙应大于 $1\mu\text{s}$ 。若为高电平，写入的为1。

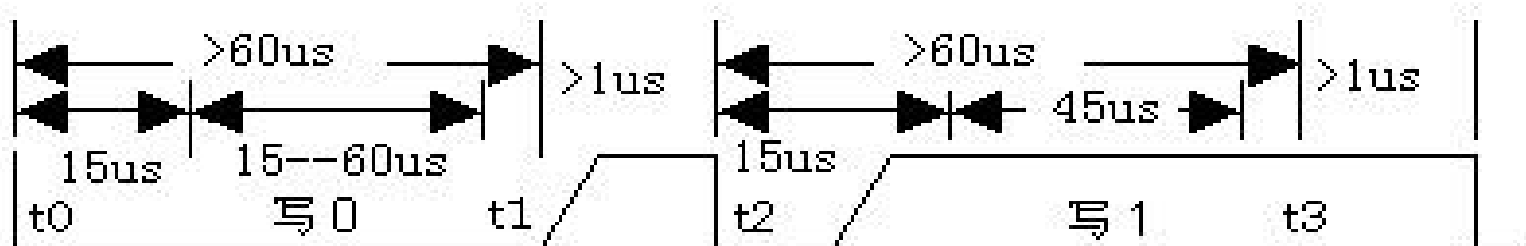


图9-21 DS18B20写时序



汇编语言程序：

DQ BIT P1.0

...

DELAY15US: MOV R7, #07H
DJNZ R7, \$
RET

DELAY60US: MOV R7, #1EH
DJNZ R7, \$
RET

WRITE_BYTE: MOV R7,#08H
(先低位, 后高位) 入DS18B20
SETB DQ
NOP
NOP

;**将累加器A中数循环写**

LOOP1:CLR DQ ; 产生15us的负脉冲

CALL DELAY15US

RRC A ; 将最低位数据移到cy

MOV DQ,C ; 将最低位数据位送数据线

CALL DELAY60US ; 产生60us的负脉冲

SETB DQ ; 数据线拉高，为写入下一位做

准备

CALL DELAY15US

DJNZ R7,LOOP1

RET

C语言程序:

```
Sbit DQ=p1^0;
```

```
void delay(unsigned int i)
```

```
{ unsigned int j;
```

```
for (j=i;j > 0;j--);
```

```
}
```

```
void wrbyte(unsigned char data)
```

```
{ uchar i;
```

```
for(i=8;i>0;i--)    /*循环写8位（先低位，后高位）*/
```

```
{ DQ=0;                /*产生15us的负脉冲*/
```

```
  delay(1);
```

```
  DQ=data&0x01;    /*将当前数据位送数据线*/
```

```
  data=data>>1;    /*将下一位数据移到最低位*/
```

```
  delay(5);        /*延时60us*/
```

```
  DQ=1;            /*数据线拉高，为写入下一位做准备*/
```

```
  delay(1);
```

```
}
```

```
}
```

3. 读时序

DS18B20读时序如图9-22，单片机在 t_0 时刻将总线从高拉至低电平，保持 $1\mu\text{s}$ 。在 t_1 时刻将总线拉高，延时 $15\mu\text{s}$ 。在 t_2 时刻读取数据，并延时至少 $45\mu\text{s}$ 。在 t_3 时刻将数据线拉高至少 $1\mu\text{s}$ ，为写入下一位数据做准备。

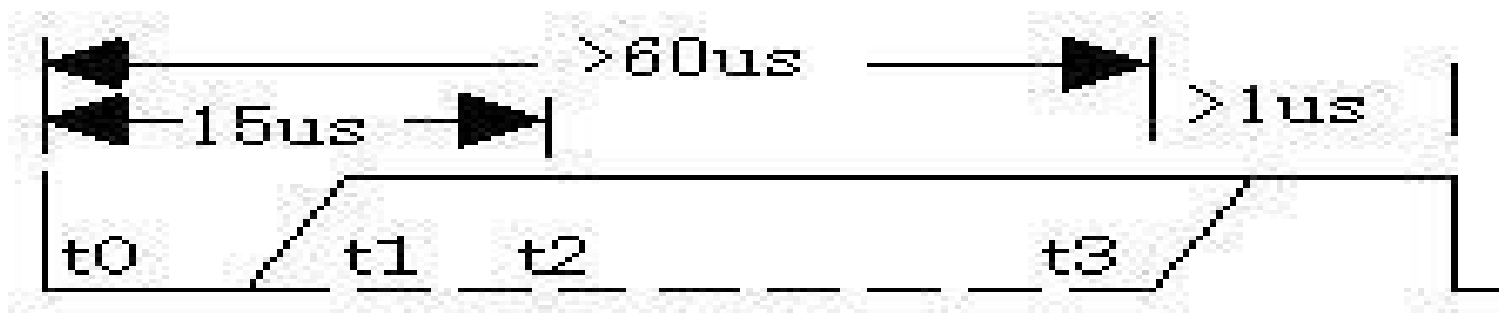


图9-22 DS18B20读时序



汇编语言程序:

DQ BIT P1.0

...

DELAY15US: MOV R7, #07H

DJNZ R7, \$

RET

DELAY60US: MOV R7, #1EH

DJNZ R7, \$

RET

READ_BYTE:MOV R7,#08H ;读DS18B20的一个字节并放在累加器A中

SETB DQ

NOP

NOP



```
LOOP:    CLR DQ
        NOP
        NOP
        NOP
        SETB DQ
        CALL    DELAY15US
        MOV C,DQ    ; 读取数据放入CY
        CALL    DELAY60US
        RRC A    ; 读出的数据右移进累加器A中
        SETB DQ
        CALL    DELAY15US
        DJNZ R7,LOOP
        CALL    DELAY60US
        RET
```




C语言程序:

Sbit DQ=P1^0;

unsigned char bdata dat

Sbit dat7=dat^7;

void delay(unsigned int i)

{

unsigned int j;

for (j=i;j > 0;j--);

}

```
unsigned char rdbyte(void)
{ unsigned char i;
  dat=0;          /*读出数据初值为0*/
  for(i=8;i>0;i--) /*循环读8位（先低位，后高位）*/

  { DQ=0;          /*产生1us的负脉冲*/
    _nop_();
    DQ=1;          /*数据总线拉高，延时15us*/
    delay(1);
    dat7=DQ;       /*读取数据*/
    delay(4);      /*延时60us */
    dat=dat>>1;    /*读出数据右移一位*/
    DQ =1;         /*数据线拉高，为写入下一位做准备*/
    _nop_();
  }
  return(dat);
}
```

9.3.4 DS18B20的操作流程及指令说明

1. DS18B20的操作流程

主机控制DS18B20完成温度转换必须经过三个步骤：每一次读写之前都要对DS18B20进行复位，复位成功后发送一条ROM指令，最后发送RAM指令，这样才能对DS18B20进行预定的操作。

2. 基本操作指令

DS18B20提供了一系列的指令来控制传感器的工作，上电后，传感器处于空闲状态，需单片机向其发送指令，控制它进行相应的操作。

(1) READ ROM [33H]

在多点温度测量系统安装前，首先必须知道每只DS18B20的产品序列号，可以将DS18B20逐个与单总线挂接，由单片机发该命令，从激光ROM读出8个字节的序列号。注意：执行该指令时，如果总线上有多个DS18B20时，所有的DS18B20会试图同时传送信号，这样就会发生数据冲突，导致操作失败。

(2) SKIP ROM (跳过ROM) [CCH]

单片机可用这一命令同时访问总线上的所有设备而不需送出ROM序列码信。例如：发出SKIP ROM命令后接着送出CONVERT命令，可使总线上的所有DS18B20同时进行温度转换；在单点系统中，此命令允许主控器不提供64位ROM编码而访问从器件以简化操作，节省时间。

(3) MATCH ROM[55H]

执行该命令后，单片机必须接着向DS18B20发送8个字节的產品序列号，以选中单总线上指定的DS18B20，只有完全匹配的DS18B20才能对随后由单片机发出的读暂存器操作命令进行响应。

(4) CONVERT T [44H]

该命令开始一次温度转换。转换结束后，数据保存在暂存器中2个字节的温度寄存器TMSB、TLSB中。

(5) READ SCRTCHPAD [BEH]

用该命令可读暂存器的内容。数据传送开始于字节0的最低位，直到暂存器的第9字节被读取。温度寄存器TLSB、TMSB处于暂存器的开始两个字节，如只需读取温度值，在读取开始的两个字节后，可用初始化命令结束读操作

9.3.5 电子温度计的设计

用数字温度传感器DS18B20构成一个电子温度计，将所测的温度值在LED显示器上显示出来，保留一位小数。硬件连接图如图9-23所示。

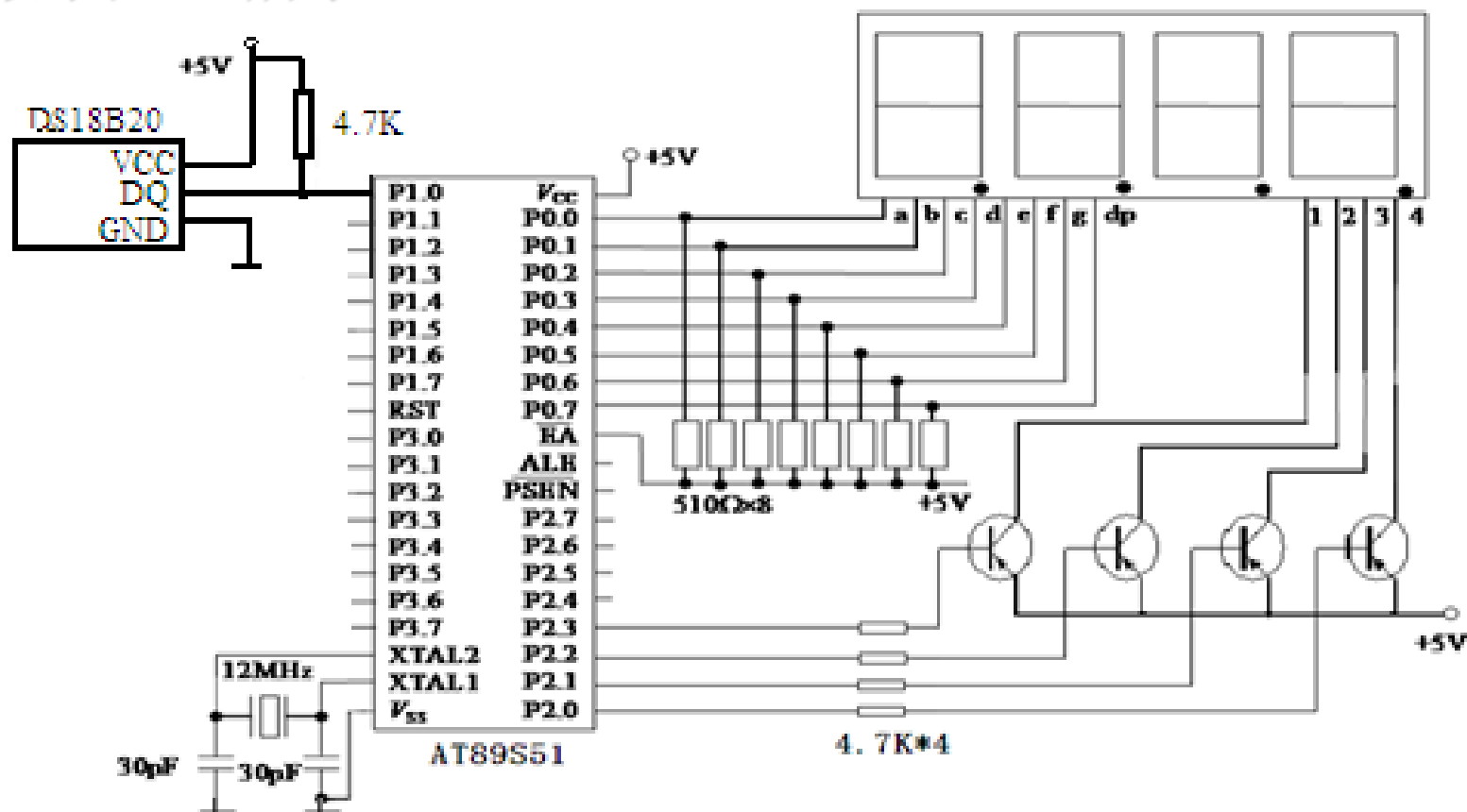


图9-23 电子温度计的硬件连接图

1. 设计思想:

- 1) 单片机首先对DS18B20进行复位操作。
- 2) 由于总线上只有一只DS18B20，单片机可用SIKP ROM[CCH]指令跳过传感器序列号识别。
- 3) CONVERT T [44H]指令启动传感器温度转换，传感器转换完成后，自动将当前温度值放入内部暂存器的开始两个字节中。
- 4) 为了读取温度值，单片机仍需对传感器进行复位操作，并跳过ROM识别，然后发读暂存器指令READ SCRTCHPAD [BEH]。
- 5) 单片机连续读出两个字节的温度值，将其转换为十进制数在数码管上显示出来。

2. 将温度值转换为十进制的方法

温度值为二个字节的十六位二进制数，见表9-17所示。高五位为符号位，高字节的低三位和低字节的高四位组成一个七位温度整数部分，最低四位为是小数部分。高字节×256+低字节即可得到一个16位二进制数，如果是负数对其求补。将这个16位二进制数右移4位去掉小数部分后，即可得到温度整数部分的真值，转换成十进制数后就是温度的百、十、个位值。

表9-17温度值值的十六位二进制数表示

温度值（℃）	输出二进制码	十六进制表示
+125	0000 0111 1101 0000	07D0H
+25.75	0000 0001 1001 1100	019CH



低字节的低四位化成十进制数后，就是温度的小数部分。小数部分只有四位，取值范围是0~F，由于只要精确到0.1℃, 可通过查表来简化这种转换，小数部分二进制数对应十进制小数数值见表9-18。

表9-18小数部分二进制数对应十进制小数数值转换表

小数部分二进制值	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
十进制值	0	0	1	1	2	3	3	4	5	5	6	6	7	8	8	9

例如，最后4位为 ‘1101’表示十六进制数 ‘C’，对应的十进制小数为0.7。

3. 软件流程图

系统由主程序、温度测量子程序、温度转换子程序和显示子程序等组成。

(1) 主程序

在主程序中首先初始化，检测DS18B20是否存在，然后通过调用读温度子程序读出DS18B20的当前值，调用温度转换子程序把从DS18B20中读出的值转换成对应的温度，调用显示子程序把温度值在数码管的相应位置进行显示。

(2) 温度转换子程序

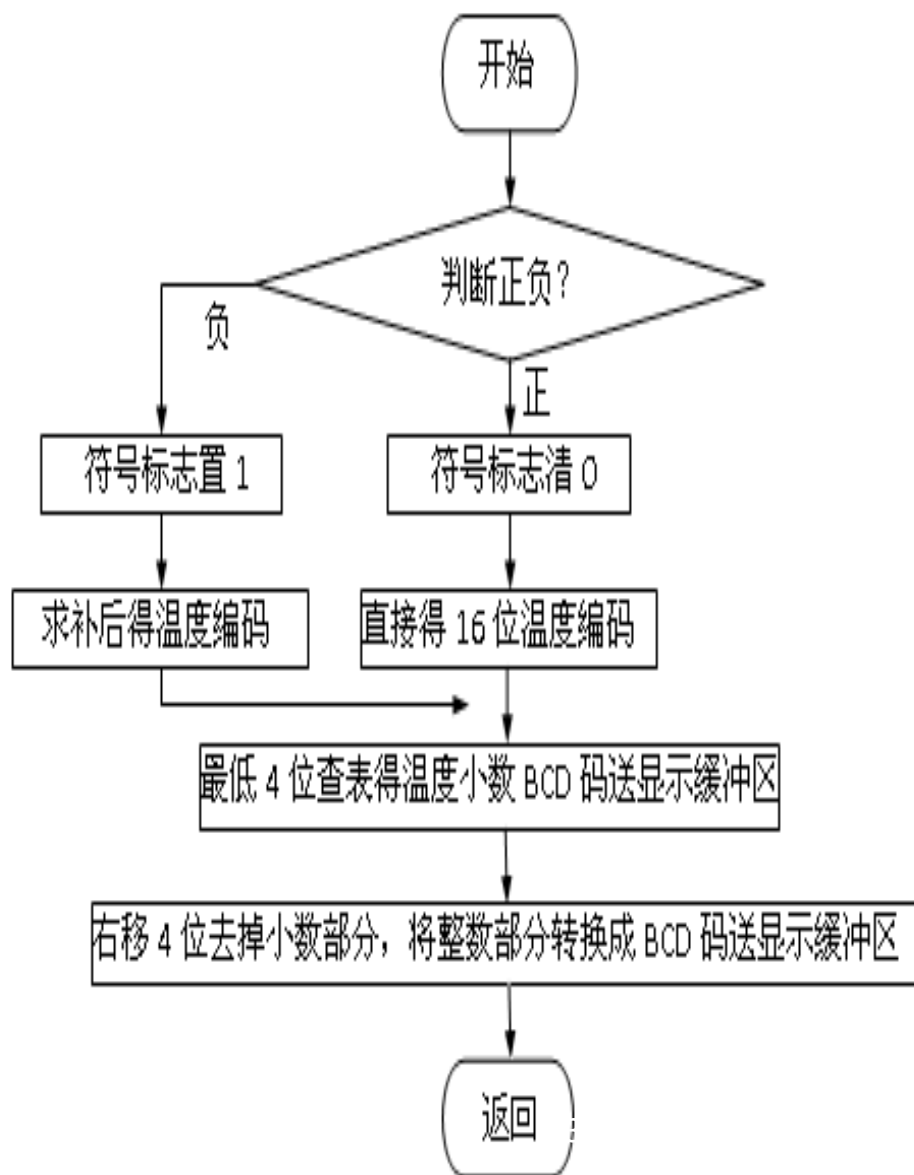
温度转换子程序的功能是发复位命令，发跳过ROM命令，发启动温度转换命令。

(3) 读取温度值

读出并处理DS18B20测量的当前温度值，读出的温度值以BCD码的形式存放在缓冲区。

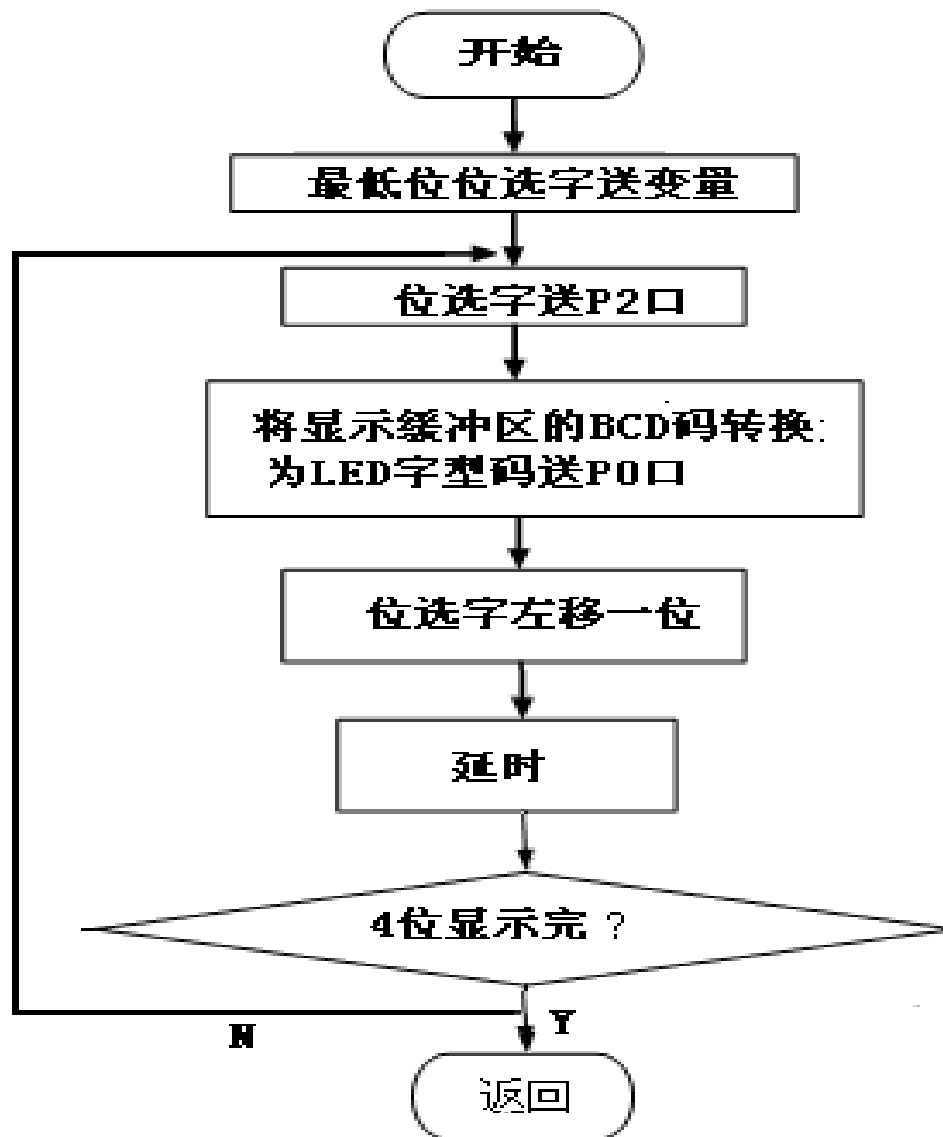
(4) 温度数据处理子程序

温度数据处理子程序流程如图9-24所示。



(5) LED动态显示子程序

LED动态显示子程序流程如图9-25所示。





4. 参考源程序见课本.



本章小结